B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Development of a Modular 3D Game Engine Framework: Core Architecture, Editor Tooling and Scene Management in C++

By
Brian Walsh

April 25, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **2**
1.0.1 Project Context and Rationale . . . 2
1.0.2 Project Objectives . . . 3
1.0.3 Technology Stack . . . 4
1.0.4 Dissertation Structure . . . 5
1.0.5 Project Repository . . . 6
1.0.6 Chapter Summary . . . 6

2 Methodology **7**
2.0.1 Development Approach . . . 7
2.0.2 Collaboration and Communication . . . 7
2.0.3 Division of Responsibilities . . . 8
2.0.4 Research and Problem-Solving Strategy . . . 8
2.0.5 Development and Testing Process . . . 9
2.0.6 Iteration and Refinement . . . 9
2.0.7 Integration Testing . . . 10
2.0.8 Methodological Reflection . . . 10

3 Technology Review **11**
3.0.1 Fixed-Timestep Game Loop Architecture . . . 11
3.0.2 Input System Design Patterns . . . 14
3.0.3 Editor Mode and Immediate-Mode GUI . . . 16
3.0.4 Raycasting and Object Selection Systems . . . 19
3.0.5 Scene Management and Serialisation . . . 21

4 System Design **24**
4.1 Architecture Overview . . . 24
4.2 Core Game Loop Implementation . . . 26
4.2.1 Fixed-Timestep Architecture . . . 27
4.2.2 Window State Management . . . 29
4.3 Input System Implementation . . . 31
4.3.1 Three-State Input Tracking . . . 31
4.3.2 GLFW Callback Integration . . . 31
4.3.3 Per-Frame State Reset . . . 33
4.3.4 ImGui Input Forwarding . . . 34
4.3.5 Mouse Delta Tracking . . . 35
4.4 Object Selection and Raycasting Implementation . . . 36
4.4.1 Screen-Space to World-Space Transformation . . . 36
4.4.2 Ray-AABB Intersection Testing . . . 38
4.4.3 Closest-Hit Selection Logic . . . 40
4.4.4 Input Coordination and Selection State . . . 41
4.4.5 Editor-Specific Selection Extensions . . . 41

4.5.1 Mode Switching Mechanism 42
4.5.2 Editor Mode Configuration 44
4.5.3 Game Mode Configuration 45
4.5.4 Interactive Transform Gizmos 45
4.5.5 Dear ImGui Integration 47
4.6 Scene Management and Serialisation 48
4.6.1 JSON Serialisation Format 49
4.6.2 GameObject Deserialisation 51
4.6.3 Editor-Specific Entity Serialisation 52
4.6.4 Object Identity and Reference Management 53
4.6.5 File System Integration 53
4.6.6 Scene Management UI Integration 54

5 System Evaluation 56
5.1 Objective 1: Fixed-Timestep Game Loop 56
5.1.1 Timing Accuracy 56
5.1.2 Spiral of Death Prevention 57
5.1.3 Editor Mode Physics Pausing 57
5.1.4 Performance and Stress Testing 58
5.2 Objective 2: Editor Mode with Interactive Debugging Tools 58
5.2.1 Mode Switching Functionality 59
5.2.2 Dear ImGui Integration 59
5.2.3 Interactive Transform Gizmos 59
5.3 Objective 3: Modular Input System with State Tracking 60
5.3.1 State Tracking Accuracy 60
5.3.2 ImGui Input Coordination 61
5.3.3 Camera Controller Integration 61
5.4 Objective 4: Component-Based Object Interaction and Selection System 61
5.4.1 Coordinate Transformation Accuracy 61
5.4.2 AABB Intersection Testing 62
5.4.3 Inspector Panel Integration 63
5.5 Objective 5: Scene and Game Object Management System 63
5.5.1 JSON Serialisation Correctness 64
5.5.2 Load/Save Cycle Integrity 64
5.5.3 Editor Workflow Integration 64
5.6 Challenges and Limitations 65
5.6.1 ImGui Ownership Refactoring 65
5.6.2 UI Click-Through Bug 65
5.6.3 Keyboard Input Conflicts 66
5.6.4 Editor/Game Mode State Management 66

6 Conclusion **67**
6.0.1 Achievement of Objectives 67
6.0.2 Insights and Lessons Learned 68
6.0.3 Limitations and Future Work 69
6.0.4 Broader Impact and Transferable Skills 69
6.0.5 Final Remarks 70

A Source Code **71**

List of Figures

3.1 Fixed timestep updates (A–E) occurring at fixed intervals independent of the variable render frame rate. Physics steps fire only when sufficient time has accumulated which decouples simulation from rendering [1]. 13

3.2 Dear ImGui demo window illustrating the range of available widgets including sliders, colour pickers and collapsible tree nodes. [2]. . . . 18

3.3 The full coordinate space transformation pipeline from local object space through world, view and clip space to final screen space. Ray picking reverses this pipeline using inverse projection and view matrices [3]. 20

3.4 Unity's Hierarchy window reflecting a scene graph with parent-child object relationships and transform inheritance [4]. 21

3.5 Separation of editor and runtime state. Physics simulation takes ownership of object transforms on entering play mode; authored transforms are restored when returning to editor mode [4]. 23

4.1 Game Engine Architecture Overview 25

4.2 Fixed-timestep game loop flowchart showing accumulator-based physics updates decoupled from rendering 29

4.3 Input system architecture showing GLFW callback integration, state tracking arrays and query API 33

4.4 Coordinate transformation pipeline for converting screen-space mouse clicks into world-space selection rays 38

4.5 Editor and game mode state diagram showing system configuration differences and E key toggle mechanism 43

4.6 Scene serialisation format showing JSON structure for game objects with transform and physics data 49

5.1 Statistics panel showing fixed-timestep performance (FPS: 60.0, Delta Time: 0.0167s) and physics state (2 rigid bodies, physics enabled) . 57

5.2 Interactive transform gizmo showing red (X), green (Y) and blue (Z) axes for axis-constrained object manipulation 60
5.3 Console output showing editor raycasting calculations with screen coordinates (484, 672), world-space origin and normalised direction vector. 62
5.4 Inspector panel displaying properties of selected object (ID: 2, Name: Player) including transform data, physics configuration and tag system 63
5.5 Scene Manager UI panel providing save, load, rename and delete operations with dropdown list of available scenes 65

Chapter 1

Introduction

The development of modern video games relies heavily on software systems known as game engines. These engines serve as the foundational framework that orchestrates real-time rendering, physics simulation, input processing and game logic into 3D interactive experiences. Commercial engines such as Unity and Unreal Engine have become industry standards. They offer powerful tools and abstractions that enable rapid game development. However, these high-level abstractions often obscure the fundamental architectural decisions, memory management strategies and real-time system constraints that underpin interactive 3D applications. By building a game engine from scratch, developers gain invaluable insights into computer graphics pipelines, physics integration, software architecture patterns and the intricate challenges of real-time systems programming.

This project presents the development of a modular 3D game engine built using C++, OpenGL, GLFW and Bullet Physics. The engine architecture is decomposed into three distinct, loosely-coupled components: a core framework, a rendering pipeline and a physics simulation system. This modular approach not only facilitates parallel development by multiple team members but also mirrors the architectural philosophies employed by professional game engines. Giving us clear separation of concerns that enables maintainability, extensibility and independent optimisation of subsystems.

1.0.1 Project Context and Rationale

The primary motivation for undertaking this project is for educational purposes. Building a game engine from scratch provides a deep understanding of what happens "under the hood" of commercial engines. Doing this reveals the complex interplay between graphics APIs, physics solvers, memory allocators and timing systems. The skills developed through this endeavor are highly transferable across various domains in software engineering, including real-time simulation systems,

scientific visualisation, robotics control systems and high-performance computing applications.

Modern game engines must balance competing demands: they must render complex 3D scenes at high frame rates (typically 60 frames per second or higher), simulate realistic physics interactions with precision, respond to user input with minimal latency and manage limited system resources efficiently. Understanding how to architect systems that meet these real-time constraints requires hands-on implementation experience that cannot be fully replicated through the use of existing engines.

This project adopts a team-based approach, with three developers each responsible for a distinct architectural component. This dissertation focuses specifically on the **core framework architecture** component. This serves as the foundational layer upon which the rendering and physics systems are built. The other two team members are independently documenting their respective components—the real-time rendering pipeline and the physics simulation integration—in their own dissertations.

1.0.2 Project Objectives

The development of the framework component is guided by five specific technical objectives, each of which addresses a critical requirement for a functional game engine architecture:

Objective 1: Implement a Robust Fixed-Timestep Game Loop

Develop a game loop that decouples physics simulation timing from rendering frame rate. The implementation must use an accumulator pattern to maintain a consistent 60 Hz physics update rate regardless of variations in rendering performance. Also there must be safeguards against a "spiral of death" scenario where frame processing time exceeds the fixed timestep duration. The system must also handle window state changes efficiently by skipping rendering when the window is minimised to conserve system resources.

Objective 2: Develop an Editor Mode with Interactive Debugging Tools

Design and implement a comprehensive editor mode that operates alongside the standard game mode. This will provide developers with runtime debugging capabilities and scene manipulation tools. The editor must integrate Dear ImGui for user interface rendering. Providing debug panels for performance statistics, physics visualisation and time tracking. The system must support interactive gizmos for object manipulation, including translation handles with axis-constrained dragging and provide visual feedback to clearly distinguish between editor and game modes during runtime.

Objective 3: Create a Modular Input System with State Tracking

Design and implement an input management system that accurately tracks keyboard and mouse input states across multiple categories (pressed, held, released). The system must integrate seamlessly with GLFW's callback-based event model while providing a simplified API for game logic. Input events must be correctly forwarded to Dear ImGui when the UI is active and the system must correctly reset per-frame state at the beginning of each frame to prevent stale input data from affecting subsequent frames. Support for camera control, object deletion and UI toggling through keyboard shortcuts must be demonstrated.

Objective 4: Design a Component-Based Object Interaction and Selection System

Develop an object selection and interaction system that enables editor-mode manipulation of scene objects through mouse-based raycasting. The system must support single and multiple object selection, provide an inspector panel for live transform editing and implement visual selection highlighting to indicate currently selected objects. Mouse ray calculations must be logged and debugged to ensure accurate 3D picking. The system must prevent UI click-through events from inadvertently selecting scene objects. Selected objects must be deletable through both inspector panel controls and keyboard shortcuts.

Objective 5: Implement a Scene and Game Object Management System

Develop infrastructure for managing the lifecycle of game objects within a scene. This will include a comprehensive save and load system that persists scene state to disk. The system must support named scene management with capabilities for creating, renaming, loading and deleting scenes exclusively within editor mode. Each game object must be assigned a unique identifier for tracking and serialisation purposes. The architecture must maintain separate transform ownership between editor and game modes, allowing physics-driven transforms during gameplay while preserving authored transforms in the editor. Scene reset functionality must correctly restore the initial state without memory leaks or leftover resources.

1.0.3 Technology Stack

The framework is implemented using a selected set of industry-standard technologies:

C++ serves as the primary programming language. Chosen for its status as the industry standard for game engine development. C++ provides the low-level memory control, performance characteristics and object-oriented design capabilities necessary for real-time systems programming.

CMake functions as the build system. Manages dependencies and generates platform-specific build configurations. CMake's cross-platform capabilities ensure

that the same build scripts work consistently across different development environments.

GLFW is used for window management and OpenGL context creation. This library abstracts platform-specific windowing system APIs and provides a consistent interface for handling window events such as keyboard input, mouse input and gamepad input across Windows, macOS, and Linux.

OpenGL serves as the graphics API. Providing low-level control over the rendering pipeline. While the framework itself does not implement rendering logic, it is responsible for initialising the OpenGL context and ensuring proper setup for the rendering module.

Bullet Physics is integrated as the physics simulation engine. This library handles collision detection, rigid body dynamics and constraint solving. The framework provides the timing infrastructure that drives physics updates while remaining agnostic to the internal workings of the physics solver.

Dear ImGui is utilised as the graphical user interface library for creating debug tools runtime configuration menus and developer interfaces. ImGui provides an immediate-mode GUI that integrates seamlessly with OpenGL, allowing the framework to expose runtime controls for object spawning, scene management and engine parameter adjustment without requiring complex UI state management or event handling systems.

GitHub is used for version control. GitHub uses a branch-based workflow that allows team members to develop features in isolation before merging them into the main codebase.

Visual Studio 2022 serves as the integrated development environment, providing debugging tools, code navigation and build management capabilities.

1.0.4 Dissertation Structure

This dissertation is organised into the following chapters:

Chapter 2: Methodology outlines the iterative development approach used throughout the project, detailing team collaboration practices, testing strategies and the research-driven process used to make architectural and implementation decisions for the framework component.

Chapter 3: Technology Review examines the academic and industry literature related to game engine architecture, real-time systems design and the specific technologies employed in this project. This chapter surveys existing approaches to game loop implementation, input system design, scene management patterns, and modular engine architectures, establishing the theoretical foundation for the design decisions made in this project.

Chapter 4: System Design provides a detailed technical explanation of the framework architecture, including comprehensive diagrams illustrating component

relationships, data flow between subsystems, and the interfaces through which the framework communicates with rendering and physics modules. This chapter documents the implementation of each major framework subsystem, explaining design decisions and their implications for engine performance and maintainability.

Chapter 5: System Evaluation assesses the framework implementation against the five objectives established in this introduction. This chapter presents performance benchmarks, timing accuracy measurements, and functional testing results that demonstrate the framework's capabilities and limitations. Any architectural trade-offs or constraints discovered during development are discussed, along with their impact on overall engine functionality.

Chapter 6: Conclusion summarises the project outcomes, reflecting on the extent to which the stated objectives were achieved and discussing insights gained through the development process. This chapter also identifies opportunities for future work and potential improvements to the framework architecture.

1.0.5 Project Repository

The complete source code for this project, including all framework components, build configurations and documentation is maintained in a GitHub repository accessible at:

`https://github.com/ATU-Game-Engine/GameEngine`

The repository contains detailed build instructions, dependency installation guides and usage examples demonstrating how to initialise and run the engine. The repository structure organises code into directories for framework, rendering, and physics components with each module maintaining its own header files and implementation files to enforce architectural boundaries.

1.0.6 Chapter Summary

This introduction has established the context and motivation for developing a modular 3D game engine from scratch. Emphasising the educational value of understanding real-time systems architecture at a fundamental level. Five specific objectives have been articulated, providing measurable criteria against which the framework implementation will be evaluated in Chapter 5. The technology stack has been justified in terms of industry relevance and technical appropriateness for real-time systems development. With this foundation established, the following chapters will delve into the research, design, implementation, and evaluation of the framework architecture.

Chapter 2

Methodology

2.0.1 Development Approach

This project required a methodical approach to software engineering that balanced the demands of real-time systems development with the collaborative nature of team-based work. The framework component that serves as the foundational infrastructure for the entire engine, had to be carefully planned and incrementally developed to ensure stability and compatibility with the rendering and physics modules being developed in parallel.

The development process followed an iterative methodology organised around development cycles. Each cycle targeted specific technical milestones that built upon previous achievements. This allows the framework to evolve from basic window initialisation and input handling toward more sophisticated features such as editor modes and scene serialisation. This incremental approach proved essential for managing complexity, as it allowed fundamental systems to be thoroughly tested before dependent features were introduced.

Progress tracking was managed through Jira, which provided task organisation, sprint planning capabilities and visibility into development timelines. Tasks were structured around deliverables—implementing the fixed-timestep game loop, integrating Dear ImGui for debug interfaces or developing the object selection system—rather than abstract goals. This ensured that each development cycle produced measurable progress toward the project objectives established in the introduction.

2.0.2 Collaboration and Communication

The team maintained a weekly in-person meetings where integration issues were discussed, architectural decisions were debated and upcoming work was planned. These meetings served as synchronisation points where each team member could

demonstrate their progress and identify dependencies that required coordination. For instance, the framework's Camera system needed to expose view and projection matrices that the rendering module could consume, requiring discussion of the appropriate interface design.

Between these structured meetings, the team communicated through a messaging board that facilitated resolution of questions, sharing of debugging insights and coordination of code integration. This lightweight communication model avoided the overhead of scheduling additional meetings while still enabling timely decision-making. The messaging channel proved particularly valuable for resolving build system issues that affected all team members, such as CMake configuration problems or dependency version mismatches.

2.0.3 Division of Responsibilities

The engine architecture was partitioned into three distinct technical domains with each team member assuming ownership of a specific component. This dissertation focuses on the framework architecture which encompasses the core engine loop, input management, camera systems, scene and object management, editor mode implementation and build system configuration. The rendering pipeline included shader programs, lighting systems, texture management and visual effects—was developed independently by another team member and is documented in a separate dissertation. Similarly, the physics simulation component—covering rigid body dynamics, collision detection and Bullet Physics integration was the responsibility of a third team member.

This division of labor enabled each developer to achieve depth of expertise in their respective domain while maintaining awareness of integration boundaries. Weekly integration tests verified that all three components could be compiled together and operated correctly. This caught interface incompatibilities before they accumulated into larger problems. The framework component's role as the integration layer made these tests particularly important, as issues in timing, object lifecycle or coordinate system transformations could manifest as failures in either rendering or physics despite originating in the framework.

2.0.4 Research and Problem-Solving Strategy

Technical decisions throughout the framework's development were informed by research into established patterns and authoritative documentation. Before implementing the fixed-timestep game loop, for example, reference materials on game engine architecture were consulted to understand the rationale for decoupling physics timing from rendering frame rate and the specific implementation details of the accumulator pattern. The design of the input system's state tracking mechanism was

influenced by analysis of how commercial engines handle per-frame input events and prevent stale state from affecting subsequent frames.

Primary sources for this research included the GLFW documentation for window and input management, C++ best practices for memory management and object lifecycle considerations and architectural references on component-based game engine design. The Dear ImGui documentation was consulted extensively when implementing the editor interface and debug panels as immediate-mode GUI differs significantly from traditional retained-mode UI frameworks.

2.0.5 Development and Testing Process

Implementation followed a test-driven approach where each framework feature was verified in isolation before being integrated into the broader system. The fixed-timestep game loop, for instance, was initially tested with simple console output to verify that physics updates occurred at the target 60 Hz frequency regardless of rendering performance variations. Only after confirming correct timing behavior was the loop integrated with actual physics simulation calls.

Visual Studio 2022 served as the primary development environment providing debugging capabilities that proved essential for diagnosing issues in real-time systems. Breakpoints were used to step through the game loop and verify accumulator state, watch windows tracked key input states across frames and performance profilers identified bottlenecks in object management or scene traversal. The IDE's CMake integration simplified the build process and ensured consistent project configuration across team members' development machines.

Editor mode features were developed with particular attention to separation of concerns between debug visualisation and core engine functionality. The debug UI panels were designed to query engine state without modifying it. This ensured that the presence or absence of debug interfaces did not affect gameplay behavior. This separation allowed debug features to be toggled on and off without impacting the physics simulation or the reproducibility of test scenarios.

2.0.6 Iteration and Refinement

The framework's development benefited from empirical testing where theoretical understanding of a feature was validated through practical implementation and observation. The raycasting system for editor object selection required understanding of coordinate space transformations, viewport-to-world space conversions and ray-object intersection mathematics. While the mathematical theory could be studied in isolation, the correct implementation required iterative debugging where ray directions were visualised, intersection points were logged and edge cases were systematically tested.

The scene serialisation system underwent multiple refinement cycles as requirements evolved. The initial implementation saved only basic transform data but testing revealed the need to serialise editor-specific state separately from runtime game state to handle object deletion without orphaning references and to maintain stable object identifiers across save/load cycles. Each refinement was driven by observed failures or limitations discovered through practical use of the system.

2.0.7 Integration Testing

Integration between the framework and the rendering and physics modules was validated through regular build-and-run tests that verified end-to-end functionality. These tests confirmed that the framework correctly initialised both subsystems, provided them with appropriate timing and state data, and coordinated their updates within the game loop. Visual inspection confirmed that the rendered output behaved as expected. Objects appeared in their correct positions, camera movement was smooth and debug visualisations accurately reflected the physics state.

Performance testing measured frame timing stability under various loads. Helping verify that the fixed-timestep accumulator maintained consistent physics updates even when rendering frame rate varied. Console logging and Dear ImGui debug panels displayed real-time metrics including physics steps per second, frame time distribution and accumulator state. This allowed performance characteristics to be monitored during development.

2.0.8 Methodological Reflection

The combination of structured iteration, validation and research-informed decision-making proved effective for developing a real-time framework component. The iterative approach allowed complexity to be managed incrementally while empirical testing ensured that theoretical understanding translated into correct implementation. Research into established patterns prevented reinvention of well-solved problems and helped avoid common pitfalls in real-time systems development. The collaborative structure with clear divisions of responsibility and regular integration testing, enabled parallel development while maintaining system coherence.

Chapter 3

Technology Review

The technology review examines the key architectural patterns, design principles and technical foundations that helped develop the framework component. Each section directly corresponds to one of the five project objectives established in the introduction and provides the theoretical and practical context necessary to understand the implementation decisions documented in subsequent chapters.

3.0.1 Fixed-Timestep Game Loop Architecture

The game loop serves as the control structure of any interactive real-time application. It orchestrates the continuous cycle of input processing, simulation updates and rendering that creates the illusion of motion and interactivity. However, the design of this loop has profound implications for the determinism, stability and performance characteristics of a game engine. The choice between variable timestep and fixed timestep architectures represents one of the most critical decisions in game engine design. The consequences can ripple through the entire system.

The Variable Timestep Problem

A naive approach to game loop implementation uses a variable timestep where the simulation advances by the exact amount of real time that has elapsed since the previous frame. This approach advances the simulation by the exact elapsed frame time (e.g. 16.7ms or 20ms). The implementation is straightforward and only requires that delta time be measured and passed to update functions.

However, variable timestep architectures introduce fundamental problems that make them unsuitable for physics simulation and gameplay systems requiring deterministic behavior. Fiedler identifies the core issue: numerical integration errors in physics simulation accumulate differently depending on timestep size. This causes identical inputs to produce different outputs when frame rates vary [5]. A

simulation that runs at 60 frames per second will produce different trajectories, collision timings and final states than the same simulation running at 30 or 120 frames per second even though the total simulated time is identical. This non-determinism makes bugs difficult to reproduce, as behaviour varies with frame rate.

Furthermore, variable timestep simulations exhibit frame rate dependent behavior. This means gameplay elements effectively run in "slow motion" at low frame rates and "fast forward" at high frame rates. While rendering may adapt gracefully to frame rate variations, physics-driven gameplay does not: a projectile fired at 30 FPS travels a different distance than one fired at 60 FPS meaning there will be a game breaking balance and making performance variations directly visible to players.

Fixed Timestep with Accumulator Pattern

The fixed timestep architecture addresses these problems by decoupling simulation updates from rendering frame rate. Rather than advancing the simulation by the variable amount of time that elapsed since the last frame, a fixed timestep loop advances simulation in consistent, fixed-size increments regardless of rendering performance. The standard implementation uses an accumulator pattern, which has become industry best practice for game engine timing [5].

The accumulator pattern maintains a running total of elapsed real time that has not yet been simulated. At the beginning of each frame, the loop measures how much real time has passed since the previous frame and adds this duration to the accumulator. The loop then enters a consumption phase, repeatedly advancing the simulation by the fixed timestep duration and subtracting that duration from the accumulator until insufficient time remains for another full step. Any remaining fractional time stays in the accumulator and carries over to the next frame.

This approach ensures that physics simulation always advances in identical increments, typically 1/60th of a second (approximately 16.67 milliseconds), producing perfectly deterministic behavior. The same inputs at the same simulated times always produce identical outputs, regardless of rendering frame rate. A simulation running at 30 FPS executes one physics step per rendered frame. Another simulation running at 120 FPS executes one physics step every two rendered frames but the physics behavior itself remains identical.

Nystrom emphasises that fixed timestep architectures are universal in commercial game engines specifically because of this determinism requirement [1]. Physics engines such as Bullet Physics and PhysX are designed with the assumption that they will be called at fixed intervals, and their integrators, constraint solvers and collision detection algorithms are tuned for specific timestep values. Violating this assumption by calling physics updates with variable timesteps degrades simulation

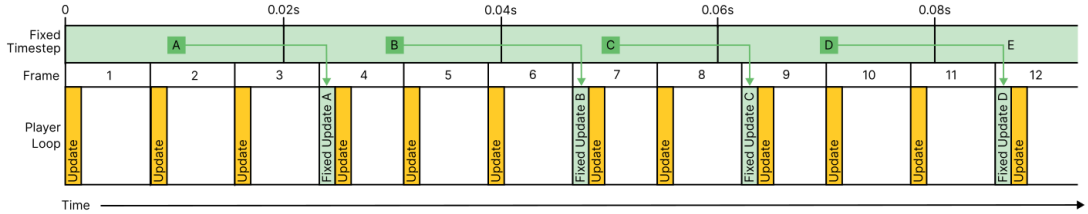

Figure 3.1: Fixed timestep updates (A–E) occurring at fixed intervals independent of the variable render frame rate. Physics steps fire only when sufficient time has accumulated which decouples simulation from rendering [1].

quality and can cause instability in constraint solving.

Handling Performance Variations

A critical consideration in fixed timestep design is the "spiral of death" scenario identified by Fiedler [5]. This occurs when a single frame takes longer to process than the fixed timestep duration. Consider a system running at 60 Hz physics (16.67ms timestep) where a particularly complex frame takes 50 milliseconds to complete. The accumulator will now contain 50ms of unprocessed time, requiring three physics steps to consume. If each of those physics steps itself takes 20ms to compute, the loop must execute 60ms worth of work. This causes the next frame to fall even further behind. The accumulator continues growing faster than the simulation can consume it, causing the application to hang as it attempts to catch up to real time.

The standard solution is to impose a maximum frame time cap, typically around 250 milliseconds. Any measured frame duration exceeding this threshold is clamped to the maximum value before being added to the accumulator [5]. This prevents the accumulator from growing unboundedly during temporary performance hitches, allowing the simulation to effectively "give up" on catching up to real time and instead skip forward, accepting that some simulated time will be lost. While this causes the simulation to run in slow motion during severe performance problems, it prevents complete application hangs and allows the system to recover once performance improves.

Alternative Approaches

While fixed timestep with accumulator has become the de facto standard, alternative approaches exist for specific use cases. Semi-fixed timestep architectures adapt the timestep duration dynamically within a constrained range. This will

allow some frame rate dependence while avoiding the worst pathologies of pure variable timestep. Subcycling approaches run physics at a higher frequency than gameplay logic this is useful when physics requires tighter integration steps than gameplay updates justify.

These alternatives introduce complexity and typically offer marginal benefits for most applications. The standard fixed timestep accumulator pattern has achieved its dominant position because it provides deterministic simulation, frame rate independence and degradation under load with relatively straightforward implementation [1]. Modern game engines including Unity, Unreal Engine and Godot all employ variations of this pattern for their core simulation loops.

3.0.2 Input System Design Patterns

Input handling represents a critical bridge between user intent and game state. It requires careful architectural design to ensure responsiveness, maintainability and correct behavior across frame boundaries. The design of an input system must address fundamental questions about event propagation, state management and the timing of input processing relative to other engine subsystems. Poor input architecture leads to dropped inputs, duplicate processing and frame-dependent behavior that degrades the player experience.

Event-Driven vs Polling Architectures

Input systems typically adopt one of two architectural paradigms: event-driven or polling-based. Event-driven systems register callback functions that the windowing system invokes immediately when input events occur, while polling systems query the current state of input devices at regular intervals during the game loop. Each approach presents distinct trade-offs in terms of timing guarantees, implementation complexity and integration with game loop architectures.

GLFW provides an event-driven callback system where the application registers functions to be invoked when keys are pressed or released [6]. This approach ensures that no input events are missed, as the operating system queues events until the application processes them through calls to `glfwPollEvents()`. Callback-based systems present integration challenges: callbacks execute outside the normal game loop flow, potentially on different threads, making it difficult to safely modify game state or physics simulation data directly from input handlers.

The polling approach queries input state during the update phase of the game loop, checking whether specific keys are currently pressed. This integrates naturally with game loop timing and guarantees that input queries occur at predictable points in the frame. However, pure polling can miss brief key presses that occur

and release between frames which will cause the game to drop inputs for fast typists or during performance hitches that extend frame duration.

State Tracking and Temporal Categories

A robust input system must distinguish between multiple categories of input state. Nystrom identifies three essential states: whether an input is currently held down, whether it was pressed this frame and whether it was released this frame [1]. These distinctions enable different gameplay mechanics: continuous movement while a key is held, single-fire actions that trigger once when pressed and release-triggered events such as charging attacks.

Tracking these states requires careful management of per-frame state transitions. The system must capture the instantaneous press and release events that occur during the polling interval, store them for query during the update phase and clear these transient states before the next frame begins. Failure to properly reset per-frame state causes pressed and released queries to return true for multiple consecutive frames and results in duplicate actions and unpredictable behavior.

The GLFW callback model provides the raw events necessary to track these states. Applications must build a state management layer on top of the callback interface [6]. Callbacks set flags indicating press and release events, the main game loop queries these flags during update and the input system resets transient flags at the beginning of each frame before polling new events.

Integration with GUI Systems

Modern game engines must coordinate input handling between game logic and user interface elements. When the player interacts with debug menus, editor interfaces, or in-game UI those interactions should not simultaneously trigger gameplay actions. An input system must therefore implement priority handling where UI elements can consume input events as to prevent them from propagating to gameplay code.

Dear ImGui provides context-based input capture. UI widgets automatically consume mouse clicks and keyboard input when the user interacts with interface elements [2]. The input system must query whether ImGui has captured the mouse or keyboard before forwarding input events to gameplay code. This will ensure that clicking on a debug panel does not simultaneously fire weapons or select objects in the game world.

This integration requires bidirectional communication: GLFW callbacks must forward events to ImGui for UI processing and the game's input system must check ImGui's capture state before processing gameplay input. The coordination

adds complexity but is essential for editor modes where debug interfaces operate alongside interactive gameplay.

3.0.3 Editor Mode and Immediate-Mode GUI

Game engines distinguish between runtime and development-time functionality through architectural separation of editor and game modes. This separation enables developers to manipulate scenes, inspect object state and debug systems without those tools interfering with gameplay behavior or being included in shipped builds. The design of editor interfaces and their integration with the core engine presents unique challenges in state management, rendering coordination and input handling.

Editor vs Runtime Architectural Patterns

Editor mode architectures must maintain clear boundaries between development tools and gameplay systems. Unity's approach separates editor code into distinct namespaces and assemblies, preventing editor functionality from being compiled into player builds [4]. This separation extends beyond code organisation to encompass state management: objects may have separate editor-time and runtime transforms which will allow developers to position entities in the scene editor while physics systems manipulate those same entities during gameplay without conflicting.

The challenge lies in managing this dual state coherently. When the engine transitions from editor to game mode, editor state must be captured as the initial conditions for gameplay with physics and other runtime systems taking ownership of object transforms and properties. Returning to editor mode requires restoring the original authored state, discarding any runtime modifications introduced during gameplay. This state ownership transfer must be explicit and deterministic to prevent subtle bugs where editor changes fail to persist or runtime state leaks back into the editor view.

Scene serialisation becomes more complex in editor contexts. When the saved representation must capture both the authored scene state and metadata about which objects are editor-only visualisations versus gameplay entities. The serialisation system must also handle references between objects correctly, maintaining stable identifiers across save and load cycles to prevent orphaned references when scenes are modified and reloaded.

Immediate-Mode GUI

Traditional GUI frameworks employ retained-mode architectures. This means the application creates and maintains a tree of widget objects that persist across frames. Event handlers update widget state and layout managers position elements. The framework tracks focus, hover states and input capture. This approach provides powerful abstractions but introduces substantial complexity in state synchronisation: the application must ensure that widget state remains consistent with underlying data, often requiring observer patterns or binding systems.

Dear ImGui adopts a fundamentally different pattern. Rather than creating persistent widget objects, ImGui applications refreshes the entire interface from scratch each frame using simple function calls [2]. A button does not persist as an object across frames. Instead, the application invokes a button function during each frame's rendering pass, providing a label and receiving a boolean value indicating whether the button was clicked in that frame.

Immediate-mode eliminates persistent widget state by removing the need to synchronise with application data. If application data changes, the next frame's ImGui code simply uses the new values and the interface automatically reflects those changes. The programming model is procedural and imperative rather than object-oriented and declarative. This aligns naturally with existing game loop structures that already regenerate rendered content each frame.

Cornut emphasises that immediate-mode GUI reduces cognitive load for developers because interface code directly expresses what should be displayed rather than maintaining a separate model of interface state [2]. The pattern particularly suits game development contexts where interfaces frequently reflect live game state and must update in real-time as that state changes. Debug panels displaying physics, memory allocation statistics, or frame timing metrics benefit from immediate-mode simplicity as they can directly query engine subsystems each frame without maintaining cached state or implementing change notification systems.

Integration with Rendering Pipelines

ImGui requires integration with the engine's rendering system, as the GUI must be drawn using the same graphics API that renders the game world. ImGui generates vertex buffers containing textured quads representing interface elements, which the renderer must submit to the GPU using appropriate render states [2]. This integration requires careful coordination of OpenGL state: the GUI rendering pass must set up orthographic projection, disable depth testing, enable alpha blending and bind the ImGui font texture without disrupting state needed by subsequent game world rendering.

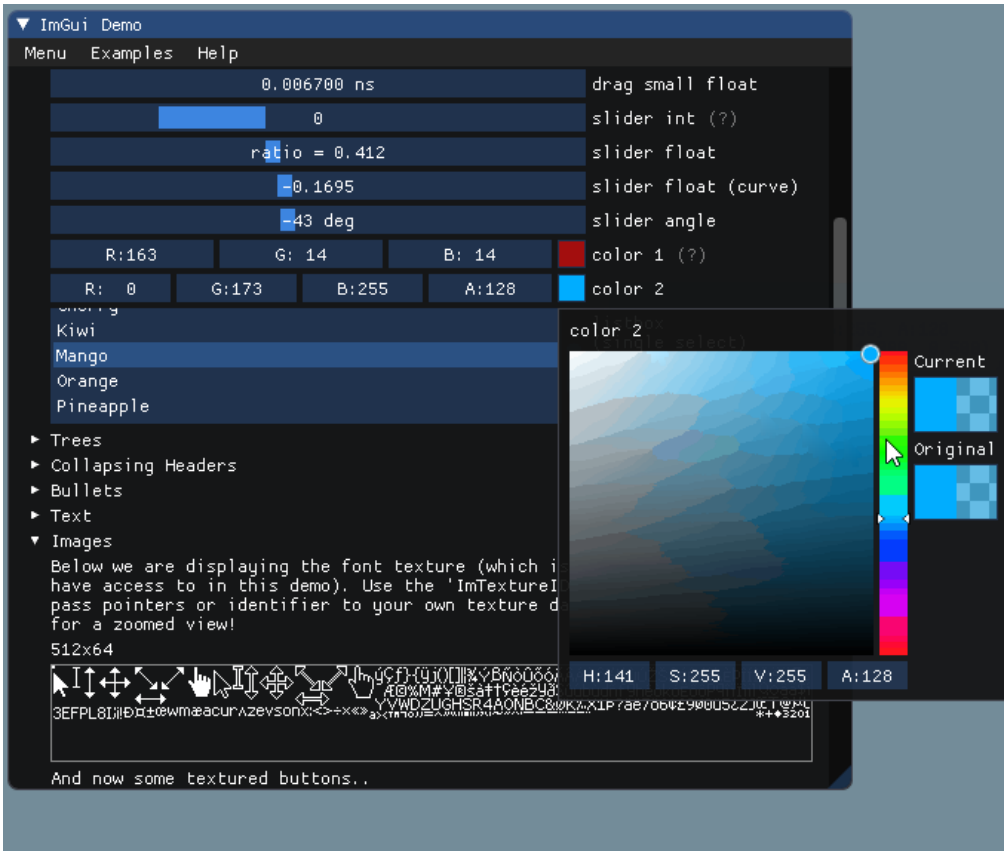

Figure 3.2: Dear ImGui demo window illustrating the range of available widgets including sliders, colour pickers and collapsible tree nodes. [2].

The rendering integration must also handle coordinate space correctly. ImGui works in pixel coordinates with origin at the top-left while 3D rendering typically uses normalised device coordinates or world space. The renderer must apply appropriate transformations and viewport configurations to ensure GUI elements appear at their intended screen positions regardless of window size or aspect ratio.

Input Capture and Event Routing

A critical aspect of editor integration is input event routing. When the user interacts with ImGui interfaces those interactions must not simultaneously trigger gameplay actions. ImGui provides query functions that indicate whether the mouse or keyboard is currently captured by interface widgets [2]. The engine's input system must check these capture states before forwarding input to gameplay code. This will ensure that clicking on a debug panel button does not also fire weapons or select objects in the game world.

This requires coordination between the windowing system, ImGui, and the input system. GLFW input callbacks must forward events to ImGui for interface processing, ImGui must update its internal capture state based on those events and the game input system must query that capture state before processing gameplay input. The coordination adds complexity but is essential for editor modes where debug interfaces operate alongside interactive gameplay.

3.0.4 Raycasting and Object Selection Systems

Editors require the ability to select and manipulate scene objects through mouse-based picking. This necessitates translating two-dimensional screen coordinates into three-dimensional world space and determining which objects intersect the resulting ray. The mathematics of raycasting, while conceptually straightforward does require careful handling of multiple coordinate space transformations and precision considerations.

Coordinate Space Transformations

A mouse click provides pixel coordinates in screen space, typically with origin at the top-left corner of the window. Converting this to a ray in world space requires reversing the transformation pipeline that renders 3D geometry to the screen. The process involves transforming from screen space to normalised device coordinates (NDC), then to view space, and finally to world space using inverse projection and view matrices [7].

Screen coordinates must first be normalised to the range [-1, 1] for both x and y axes, accounting for the aspect ratio and viewport dimensions. The resulting NDC coordinates represent a point on the near clipping plane of the view frustum. A second point on the far clipping plane creates a line segment that when transformed to world space using the inverse of the view-projection matrix, defines the picking ray's origin and direction.

Scratchapixel emphasises the importance of proper normalisation of the resulting direction vector and handling of the homogeneous w coordinate introduced by perspective projection [3]. Failure to correctly apply these transformations results in rays that diverge from the mouse cursor position or become increasingly inaccurate at screen edges where perspective distortion is greatest.

Ray-Object Intersection Testing

Once the ray exists in world space, intersection testing determines which objects the ray passes through. For axis-aligned bounding boxes (AABBs), the ray-box intersection algorithm checks whether the ray enters and exits the box within a

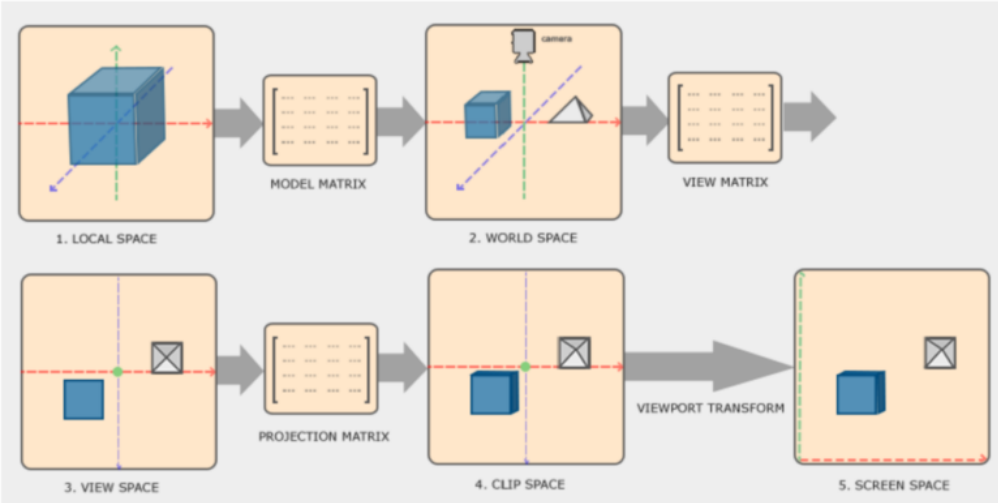

Figure 3.3: The full coordinate space transformation pipeline from local object space through world, view and clip space to final screen space. Ray picking reverses this pipeline using inverse projection and view matrices [3].

valid range [7]. More complex geometry requires testing against individual triangles or using spatial acceleration structures to avoid testing every polygon in the scene.

The intersection test returns not only whether an intersection occurred but also the distance along the ray where it occurred. When multiple objects intersect the ray, the closest intersection determines which object the user clicked. This requires maintaining the nearest intersection distance found so far and only updating the selected object when a closer intersection is discovered.

Editor-Specific Considerations

Selection systems in editor contexts must handle additional complexities beyond simple intersection testing. Gizmos and debug visualisation geometry should be selectable but with different selection behavior than gameplay objects. The system must distinguish between selecting an object's transform gizmo to manipulate it versus selecting the object itself to inspect its properties.

Multiple selection introduces further complexity: holding modifier keys should add objects to the current selection rather than replacing it and selection boxes that encompass multiple objects require frustum-based testing rather than single-ray intersection. The selection system must also coordinate with the rendering system to provide visual feedback, highlighting selected objects and rendering transform gizmos at their positions.

3.0.5 Scene Management and Serialisation

Game engines require persistent storage of scene data. Developers need to be able to save content and reload it across development sessions. Scene management encompasses object lifecycle, hierarchical relationships and serialisation to persistent formats. The architecture must balance human readability for version control with efficient parsing and maintain referential integrity as scenes are modified.

Scene Graph Architecture

Scene graphs organise game objects hierarchically with parent-child relationships defining coordinate space inheritance and lifetime dependencies. Nystrom describes the component pattern as fundamental to modern scene organisation where game objects serve as containers for behavior components rather than inheriting functionality through class hierarchies [1]. This composition-based approach enables flexible object construction: a game object becomes a light source by attaching a light component or gains physics behavior through a rigidbody component.

Unity's GameObject architecture exemplifies this pattern, providing a minimal container with transform data and a list of attached components [4]. The scene graph forms a hierarchical tree of objects, enabling traversal for rendering, physics updates and serialisation.

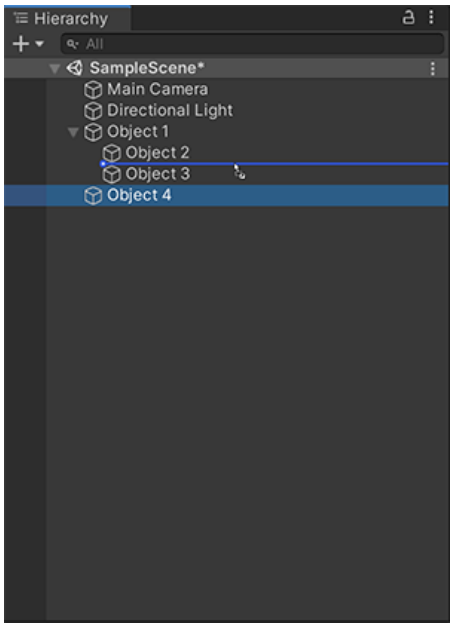

Figure 3.4: Unity's Hierarchy window reflecting a scene graph with parent-child object relationships and transform inheritance [4].

Serialisation Format Selection

Scene data must serialise to a format that balances competing requirements: human readability for version control and debugging, parsing efficiency, and expressiveness for complex object graphs with cross-references. JSON has become a de facto standard for game engine serialisation due to its text-based format, widespread tooling support, and ability to represent nested structures naturally [8].

The JSON specification provides object notation for key-value pairs, arrays for ordered collections and primitive types for numbers and strings [8]. Game object serialisation maps naturally to JSON objects with component data nested within parent object structures. Arrays represent lists of children or component collections while string identifiers enable cross-references between objects.

Reference Management and Object Identity

A critical challenge in scene serialisation is maintaining stable object identities across save and load cycles. When object A references object B, the serialisation must capture this relationship in a way that survives file round-tripping. Assigning each object a unique identifier enables reference serialisation: rather than storing direct memory pointers, the system stores the referenced object's ID which can be resolved after deserialisation by looking up the corresponding loaded object.

Reference resolution requires two-pass loading: first pass creates all objects and assigns them their IDs, second pass resolves references by matching stored IDs to loaded objects. This prevents issues with forward references where an object references another that has not yet been loaded. The ID assignment system must guarantee uniqueness and stability, typically using incrementing integers or GUIDs.

Editor State and Runtime Separation

Editor-authored scenes contain metadata that should not persist into runtime gameplay. Debug visualisation settings, editor camera positions and gizmo configurations are editor-specific state that must be filtered during scene export for shipped builds. The serialisation system must distinguish between editor-only data and runtime data. This is done either through separate files or tagged fields within a unified scene format.

Transform ownership presents a particular challenge: in editor mode, authored transforms represent the "ground truth" state, while in runtime mode, physics simulation controls object positions. The scene save system must capture editor-authored transforms while discarding any runtime physics modifications. This will ensure that returning to editor mode restores the original positions rather than wherever physics simulation left objects.

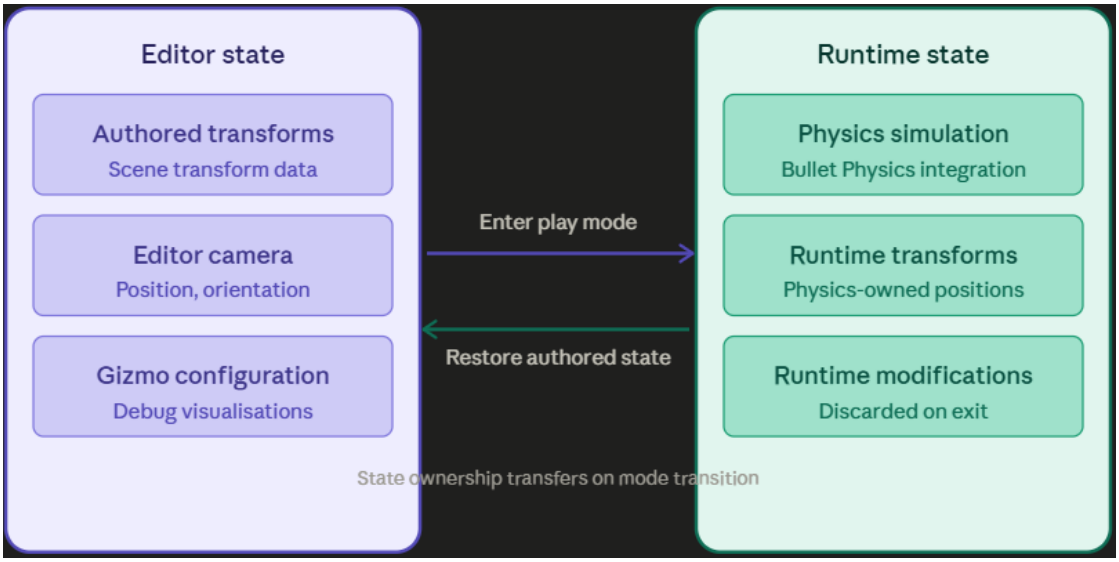

Figure 3.5: Separation of editor and runtime state. Physics simulation takes ownership of object transforms on entering play mode; authored transforms are restored when returning to editor mode [4].

Chapter 4

System Design

This chapter provides a detailed technical explanation of the framework architecture. It documents the implementation of each major subsystem and the design decisions that shaped their development. The framework serves as the foundational layer of the game engine, coordinating the rendering and physics modules whilst providing essential infrastructure for real-time interaction.

4.1 Architecture Overview

The game engine architecture follows a modular design philosophy where three distinct components—framework, rendering and physics—operate as loosely-coupled subsystems with well-defined interfaces. This architectural approach enables parallel development by multiple team members, facilitates independent testing and optimisation of each module. It also mirrors the design patterns employed by commercial game engines.

Figure 4.1 illustrates the overall system architecture. It shows the framework component at the centre, the rendering and physics modules as peripheral subsystems and the external libraries upon which the engine depends. The framework assumes responsibility for orchestrating the interaction between these components. It provides timing infrastructure, manages shared game state, and exposes development tools through an integrated editor interface.

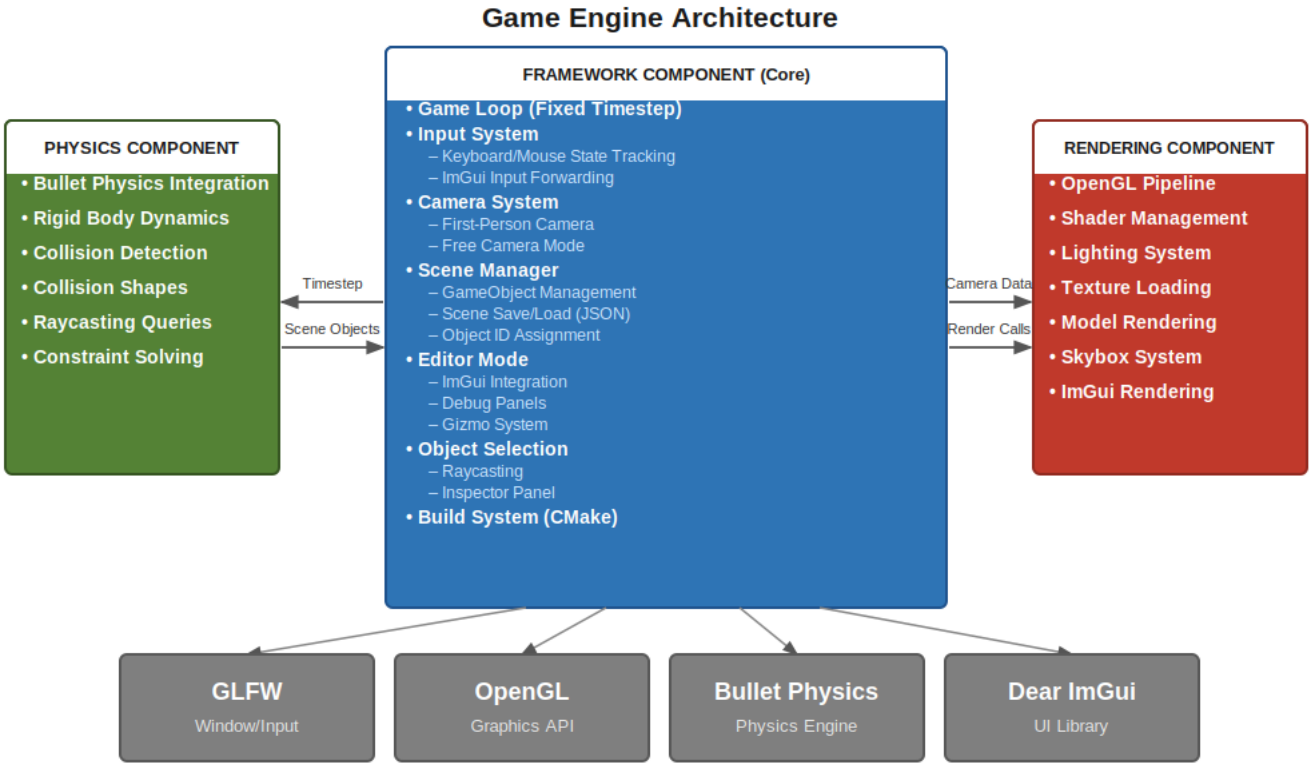

Figure 4.1: Game Engine Architecture Overview

The framework component encompasses several distinct subsystems, each addressing a specific aspect of engine infrastructure. The game loop implements the fixed-timestep accumulator pattern discussed in the technology review. This pattern ensures deterministic physics simulation independent of rendering frame rate. The input system manages keyboard and mouse state tracking, forwarding events to both gameplay code and the Dear ImGui interface. The camera system provides first-person and free-camera modes for scene navigation during both gameplay and editor sessions.

Scene management infrastructure maintains the hierarchy of game objects within the active scene. A unique identifier is assigned for serialisation and provides save/load functionality using JSON as the persistent storage format. The editor mode subsystem integrates Dear ImGui to provide runtime debugging interfaces, performance monitoring panels and interactive gizmos for object manipulation.

The object selection system implements mouse-based raycasting to enable editor-mode picking of scene entities, translating screen-space coordinates to world-space rays and performing intersection tests against object bounding volumes.

The rendering component, developed by another team member, consumes data provided by the framework—specifically camera transformation matrices and the list of renderable objects in the current scene and produces visual output using the OpenGL graphics API. The framework initialises the OpenGL context and manages the rendering window through GLFW but delegates all shader compilation, texture loading, lighting calculations and draw call submission to the rendering module.

The physics component, also developed by another team member, receives fixed-timestep updates from the framework's game loop and maintains its own representation of scene objects as Bullet Physics rigid bodies. The framework provides the physics module with timing information and notifies it when objects are created or destroyed but does not directly manipulate physics state. Transform synchronisation occurs through the scene management system: during gameplay, physics simulation controls object positions, whilst in editor mode, authored transforms take precedence.

External library dependencies form the foundation upon which all three engine components are built. GLFW provides platform-abstraction for window management and input event handling. OpenGL serves as the low-level graphics API, accessed by the rendering module but initialised by the framework. Bullet Physics implements the collision detection and rigid body dynamics algorithms used by the physics module. Dear ImGui provides the immediate-mode GUI framework integrated into both the framework and rendering components to produce the editor interface.

This architectural foundation, with the framework serving as the integration layer between specialised rendering and physics modules, provides the structure necessary to implement the five objectives established in the introduction. The following sections document the implementation details of each framework subsystem, explaining the design decisions and technical approaches that realise this architecture in executable code.

4.2 Core Game Loop Implementation

The game loop represents the beating heart of the engine. It executes continuously from application startup until termination and orchestrating the sequence of operations that produce real-time behavior. The implementation realizes the fixed-timestep accumulator pattern discussed in the technology review, ensuring deterministic physics simulation whilst accommodating variable rendering frame

rates.

4.2.1 Fixed-Timestep Architecture

The loop maintains several critical state variables that govern its operation. A high-resolution clock provided by C++ chrono facilities tracks elapsed time between frames with microsecond precision. An accumulator variable stores fractional timesteps that have not yet been simulated and a fixed timestep constant defines the granularity of physics updates at 60 Hz (16.67 milliseconds per step).

```
using clock = std::chrono::high_resolution_clock;
auto currentTime = clock::now();
double t = 0.0;                     // Total simulated time
const double dt = 1.0 / 60.0;       // Fixed timestep: 60 FPS
double accumulator = 0.0;            // Unprocessed real time
```

At the beginning of each iteration, the loop measures the duration of the previous frame by comparing the current time against a stored timestamp from the last iteration. This measured frame time represents the amount of real-world time that elapsed whilst the previous frame was processing. The chrono library's `duration` type provides precise measurements that are then converted to seconds as a double-precision floating-point value.

```
auto newTime = clock::now();
std::chrono::duration<double> frameDuration = newTime - currentTime;
double frameTime = frameDuration.count();
currentTime = newTime;
```

The implementation includes an essential safeguard against the spiral of death scenario discussed in the technology review. Any frame time exceeding 250 milliseconds is clamped to that maximum value before being added to the accumulator. This prevents the accumulator from growing unboundedly during temporary performance hitches, allowing the simulation to skip forward rather than attempting to catch up indefinitely.

```
if (frameTime > 0.25)
    frameTime = 0.25;  // Prevent spiral of death
accumulator += frameTime;
```

The consumption phase follows accumulator updates. A while loop repeatedly checks whether the accumulator contains sufficient time for another physics step. If the accumulator exceeds the fixed timestep duration, the loop invokes the physics

simulation using the fixed timestep value. It then subtracts that duration from the accumulator and increments the total simulated time. This process continues until the accumulator contains less than one full timestep. Any remaining fractional time is carried forward to the next frame.

```
while (accumulator >= dt)
{
    simulate(dt);
    accumulator -= dt;
    t += dt;
    physicsSteps++;
}
```

This architecture ensures that physics always advances in identical increments. A frame that takes 8 milliseconds to render adds 8ms to the accumulator, insufficient for a full 16.67ms step, so no physics update occurs that frame. The next frame, taking another 8ms, brings the accumulator to 16ms total, still insufficient. The third frame pushes the accumulator past 16.67ms, triggering a single physics step and leaving approximately 0.33ms in the accumulator. Over multiple frames, physics executes at exactly 60 Hz regardless of rendering performance variations.

The implementation includes diagnostic counters that track the number of physics steps executed per second, providing runtime verification that the fixed-timestep mechanism operates correctly. During typical operation at 60 FPS rendering, the system logs exactly 60 physics steps per second, confirming deterministic simulation timing.

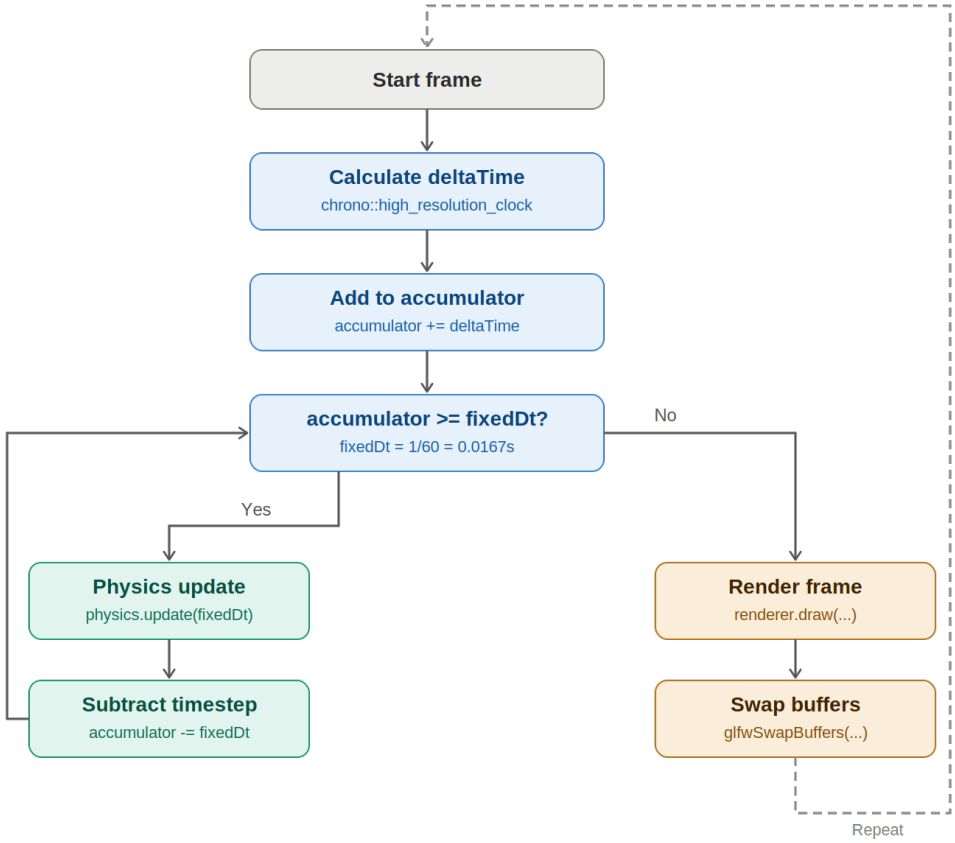

Figure 4.2: Fixed-timestep game loop flowchart showing accumulator-based physics updates decoupled from rendering

4.2.2 Window State Management

The game loop integrates comprehensive window state tracking to enable adaptive behaviour based on the application's window status. The system monitors focus state, minimisation status, window dimensions and framebuffer size. It detects changes from frame to frame and enables appropriate responses.

Window state variables are initialised before entering the main loop. The system queries GLFW for the current window attributes. The focus state indicates whether the window is the active foreground application. The minimisation state tracks whether the window has been iconified. Window size represents the logical dimensions in screen coordinates. Framebuffer size represents the actual pixel dimensions. These may differ on high-DPI displays where multiple pixels correspond to a single screen coordinate point.

```
bool isFocused = glfwGetWindowAttrib(window, GLFW_FOCUSED) == GLFW_TRUE;
bool isMinimized = glfwGetWindowAttrib(window, GLFW_ICONIFIED) == GLFW_TRUE;
int winW = 0, winH = 0;    // Window size (screen coordinates)
int fbW = 0, fbH = 0;      // Framebuffer size (pixels)
glfwGetWindowSize(window, &winW, &winH);
glfwGetFramebufferSize(window, &fbW, &fbH);
```

The system maintains shadow copies of these state variables from the previous frame, enabling change detection through comparison. Each iteration polls GLFW for current window state and compares against the stored previous values. When changes are detected, the system can trigger appropriate responses—for instance, notifying the renderer of viewport dimension changes or pausing simulation when the window loses focus.

```
if (isFocused != lastFocused) {
    lastFocused = isFocused;
    // Could potentially pause when window loses focus
}
if (isMinimized != lastMinimized) {
    lastMinimized = isMinimized;
    // Could skip rendering when minimized
}
```

The distinction between window size and framebuffer size is essential for correct rendering on high-resolution displays. On standard displays, these values match. For example, a 1920×1080 window has a 1920×1080 framebuffer. On Retina displays or Windows systems with display scaling enabled, a 1920×1080 window may instead have a 3840×2160 framebuffer. In this case, each logical point corresponds to multiple physical pixels. The framebuffer dimensions are passed to the rendering module to ensure the viewport configuration matches the actual pixel resolution.

Window state tracking also enables performance optimisations. When the window is minimised, rendering becomes unnecessary because no output is visible to the user. The framework can skip rendering calls entirely in this state, conserving GPU resources and battery power on mobile platforms. When the window loses focus, gameplay can be paused in single-player contexts. This prevents the simulation from advancing while the player's attention is elsewhere.

The integration of window state tracking with the fixed-timestep loop ensures that timing remains correct even when the window state changes. Minimising the window does not affect accumulator behaviour or physics update frequency. The loop continues processing at 60 Hz regardless of whether rendering output is visible. This separation ensures that window management concerns do not compromise simulation determinism.

4.3 Input System Implementation

The input system provides the interface between hardware input devices and engine subsystems, translating raw keyboard and mouse events into queryable state that gameplay code and editor tools can consume. The implementation addresses the challenges of callback-based event models, temporal state tracking and coordination with immediate-mode GUI systems.

4.3.1 Three-State Input Tracking

The system maintains three distinct boolean arrays that track keyboard state across temporal categories. The `keysDown` array indicates which keys are currently being held, whilst `keysPressed` and `keysReleased` track single-frame transition events. This separation enables different gameplay mechanics: continuous movement whilst a key remains held, single-fire actions triggered once on press and release-triggered events such as charging attacks.

```
static bool keysDown[1024] = { false };      // Currently held
static bool keysPressed[1024] = { false };   // Pressed this frame
static bool keysReleased[1024] = { false };  // Released this frame
```

Mouse buttons employ an identical tracking mechanism with separate arrays for down, pressed and released states. Arrays of size 1024 accommodate the full range of GLFW key codes, whilst mouse button arrays use size 8 to cover standard mouse configurations.

4.3.2 GLFW Callback Integration

GLFW operates on a callback-based event model where the windowing system invokes registered functions whenever input state changes. The `KeyCallback` function serves as the bridge between GLFW and the engine's input state, updating the three state arrays whenever a key press or release event occurs.

```
static void KeyCallback(GLFWwindow* window, int key,
                        int scancode, int action, int mods)
{
    // Forward to ImGui first
    ImGui_ImplGlfw_KeyCallback(window, key, scancode, action, mods);

    if (key < 0 || key >= 1024) return;

    if (action == GLFW_PRESS)
```

```
    {
        if (!keysDown[key])
            keysPressed[key] = true;  // First press only
        keysDown[key] = true;
    }
    else if (action == GLFW_RELEASE)
    {
        keysDown[key] = false;
        keysReleased[key] = true;
    }
}
```

The callback implementation includes an important guard: the `keysPressed` flag is only set if the key was not already down. This prevents key repeat events—where the operating system generates multiple press events whilst a key is held—from incorrectly registering as new presses each frame. Without this check, holding the space bar would trigger jump actions repeatedly rather than once.

Registration of callbacks occurs during input system initialisation, where GLFW is instructed to invoke these functions whenever events occur on the specified window. The implementation registers handlers for keyboard input, mouse movement, mouse buttons, scroll wheel and character input.

```
void Input::Initialize(GLFWwindow* window)
{
    internalWindow = window;
    glfwSetKeyCallback(window, KeyCallback);
    glfwSetCursorPosCallback(window, CursorPosCallback);
    glfwSetMouseButtonCallback(window, MouseButtonCallback);
    glfwSetScrollCallback(window, ScrollCallback);
    glfwSetCharCallback(window, CharCallback);
}
```

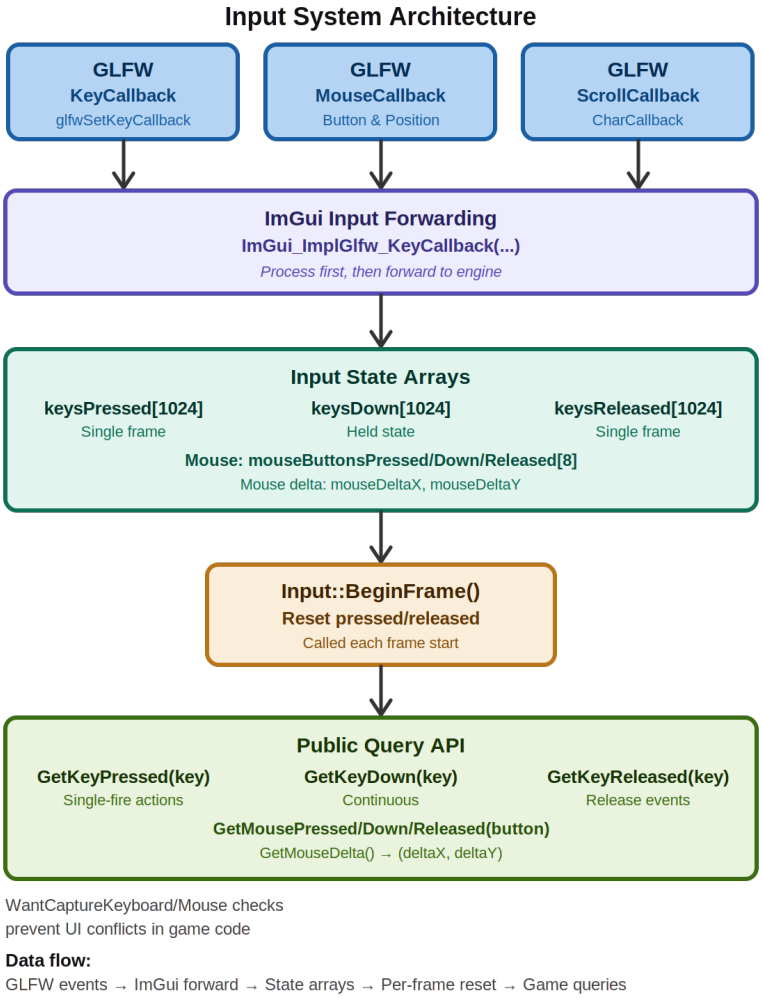

Figure 4.3: Input system architecture showing GLFW callback integration, state tracking arrays and query API

4.3.3 Per-Frame State Reset

The pressed and released states represent events that should only register true for a single frame. If these states were not cleared, a key press would continue returning true every frame until another key press overwrote it. This would cause actions to trigger repeatedly and creating unpredictable behaviour.

The `BeginFrame` function addresses this by resetting all pressed and released flags to false at the beginning of each frame, before GLFW polls for new events. The held state (`keysDown`) persists across frames, accurately reflecting whether a key remains physically pressed.

```
void Input::BeginFrame()
{
    for (int i = 0; i < 1024; i++)
    {
        keysPressed[i] = false;
        keysReleased[i] = false;
    }

    for (int i = 0; i < 8; i++)
    {
        mouseButtonsPressed[i] = false;
        mouseButtonsReleased[i] = false;
    }

    mouseDeltaX = 0.0;
    mouseDeltaY = 0.0;
}
```

This function is called at the start of the game loop, immediately after time updates but before polling GLFW events. The ordering ensures that transient states are cleared before new events arrive which will prevent stale state from affecting the current frame.

4.3.4 ImGui Input Forwarding

Integration with Dear ImGui requires bidirectional input coordination. When the user interacts with UI elements—clicking buttons, typing in text fields or dragging sliders—those interactions must not simultaneously trigger gameplay actions. Conversely, when UI elements are not active, input should flow freely to gameplay code.

The solution involves forwarding all GLFW callbacks to ImGui's input handlers before processing engine-specific logic. Each callback function begins by invoking the corresponding ImGui handler, allowing the GUI system to capture and process events.

```
static void KeyCallback(GLFWwindow* window, int key,
                        int scancode, int action, int mods)
{
    ImGui_ImplGlfw_KeyCallback(window, key, scancode,
                               action, mods);
    // Then process for game...
}
```

ImGui maintains internal state tracking which widgets are hovered, focused or capturing input. The `WantCaptureKeyboard` and `WantCaptureMouse` flags indicate whether ImGui consumed the input this frame. The game loop queries these flags before processing gameplay input, preventing click-through when interacting with UI panels.

```
if (!ImGui::GetIO().WantCaptureKeyboard &&
    Input::GetKeyPressed(GLFW_KEY_E))
{
    // Toggle editor/game mode
}
```

This pattern appears throughout the input processing code, ensuring that UI interactions remain isolated from gameplay actions. When the inspector panel is visible and the user clicks on a slider, the click does not also trigger object selection in the 3D view.

4.3.5 Mouse Delta Tracking

The camera controller requires mouse movement delta—the change in position since the last frame—rather than absolute screen coordinates. The `CursorPosCallback` maintains shadow copies of the previous frame's mouse position and calculates the delta each time the cursor moves.

```
static void CursorPosCallback(GLFWwindow* window,
                              double xpos, double ypos)
{
    ImGui_ImplGlfw_CursorPosCallback(window, xpos, ypos);

    if (firstMouse)
    {
        lastMouseX = xpos;
        lastMouseY = ypos;
        firstMouse = false;
        return;
    }

    mouseDeltaX = xpos - lastMouseX;
    mouseDeltaY = ypos - lastMouseY;

    lastMouseX = xpos;
```

```
    lastMouseY = ypos;
}
```

The first mouse movement after initialisation requires special handling to prevent a large erroneous delta. Without this guard, the delta would be calculated from uninitialised position values to the actual cursor location. This would cause the camera to snap abruptly when mouse movement first occurs.

The camera controller integration respects ImGui capture state. Mouse movement is only forwarded when the UI is not capturing input and the right mouse button is held. This enables editor-style camera control. Right-click-and-drag rotates the view, while left-click remains available for object selection.

4.4 Object Selection and Raycasting Implementation

Editor-mode manipulation of scene objects requires translating two-dimensional screen coordinates into three-dimensional world space to determine which object the user clicked. The implementation performs coordinate space transformations to generate a picking ray and tests that ray against object bounding volumes to identify intersections.

4.4.1 Screen-Space to World-Space Transformation

A mouse click provides pixel coordinates within the framebuffer, with the origin at the top-left corner. Converting this into a ray in world space requires reversing the transformation pipeline that projects 3D geometry onto the 2D screen. This process involves multiple coordinate space transformations. These include converting screen pixels to normalised device coordinates (NDC), NDC to clip space, clip space to eye space, and finally eye space to world space.

The first transformation normalises pixel coordinates to the range [-1, 1] for both the x and y axes. This accounts for the framebuffer dimensions. The y-coordinate is inverted during this step, as screen space places the origin at the top-left, whereas NDC places it at the bottom-left.

```
double mouseX, mouseY;
glfwGetCursorPos(window, &mouseX, &mouseY);

float x = (2.0f * static_cast<float>(mouseX)) / fbW - 1.0f;
float y = 1.0f - (2.0f * static_cast<float>(mouseY)) / fbH;
```

These normalised coordinates define a point on the near clipping plane of the view frustum. A four-component vector in clip space is constructed with z = -1.0 (near plane) and w = 1.0 (indicating a position rather than a direction).

```
glm::vec4 rayClip(x, y, -1.0f, 1.0f);
```

The transformation to eye space applies the inverse projection matrix, converting from the perspective-distorted clip space to the camera's local coordinate system. The resulting vector is then modified to point forward (z = -1.0) and converted to a direction (w = 0.0).

```
glm::mat4 projection =
    camera.getProjectionMatrix(static_cast<float>(fbW) / fbH);

glm::vec4 rayEye = glm::inverse(projection) * rayClip;
rayEye = glm::vec4(rayEye.x, rayEye.y, -1.0f, 0.0f);
```

The final transformation to world space applies the inverse view matrix, converting from the camera's local space to the shared world coordinate system. The result is normalised to produce a unit-length direction vector.

```
glm::vec3 rayDirection =
    glm::normalize(glm::vec3(glm::inverse(camera.getViewMatrix())
                             * rayEye));
glm::vec3 rayOrigin = camera.getPosition();
```

This multi-stage transformation correctly handles perspective projection, camera orientation and viewport dimensions. Which produces a ray that extends from the camera position through the point on the screen where the user clicked.

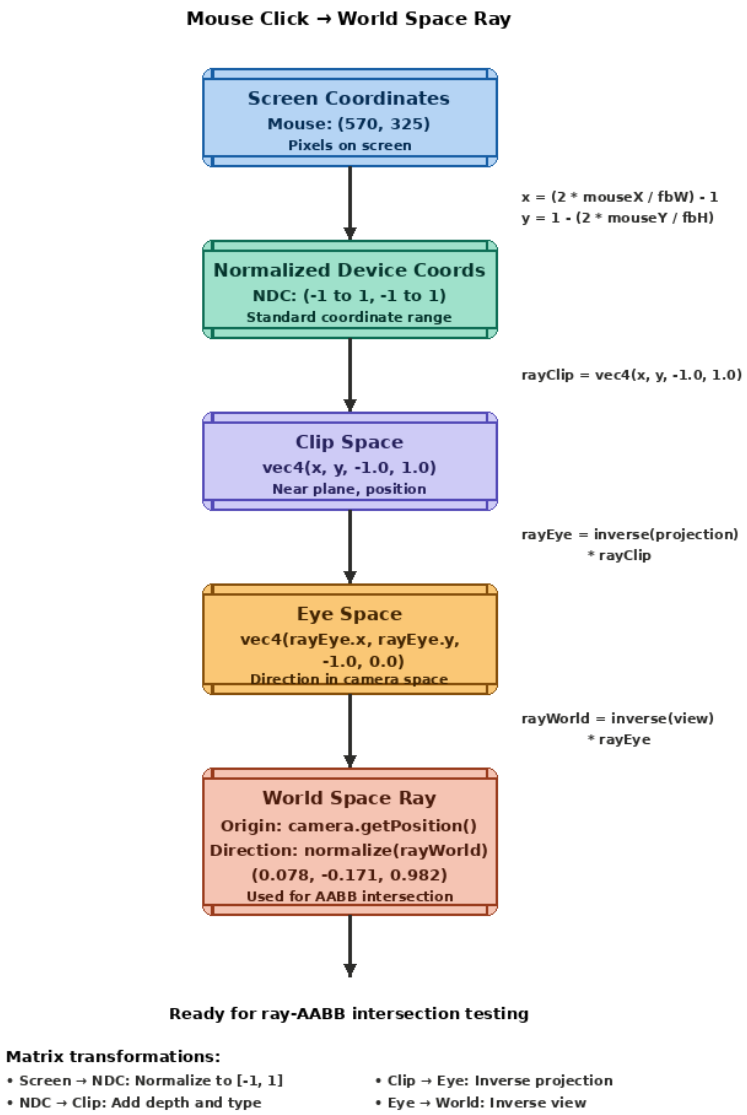

Figure 4.4: Coordinate transformation pipeline for converting screen-space mouse clicks into world-space selection rays

4.4.2 Ray-AABB Intersection Testing

Object selection uses axis-aligned bounding boxes (AABBs) as basic collision shapes. Each object's AABB is constructed from its world position and scale, assuming the object's origin lies at its centre.

```
const glm::vec3& position = obj->getPosition();
```

```
const glm::vec3& scale = obj->getScale();

glm::vec3 aabbMin = position - scale * 0.5f;
glm::vec3 aabbMax = position + scale * 0.5f;
```

The ray-AABB intersection algorithm employs the slab method. This method tests the ray against pairs of axis-aligned planes that bound the box. For each axis (x, y, z), the algorithm computes the distances along the ray where it enters and exits the slab defined by the minimum and maximum planes perpendicular to that axis.

```
bool RayIntersectsAABB(
    const glm::vec3& rayOrigin,
    const glm::vec3& rayDir,
    const glm::vec3& aabbMin,
    const glm::vec3& aabbMax,
    float& outDistance)
{
    float tMin = 0.0f;
    float tMax = FLT_MAX;

    for (int axis = 0; axis < 3; axis++)
    {
        if (std::abs(rayDir[axis]) < 0.0001f)
        {
            // Ray parallel to slab - check if inside
            if (rayOrigin[axis] < aabbMin[axis] ||
                rayOrigin[axis] > aabbMax[axis])
                return false;
        }
        else
        {
            float invDir = 1.0f / rayDir[axis];
            float t0 = (aabbMin[axis] - rayOrigin[axis]) * invDir;
            float t1 = (aabbMax[axis] - rayOrigin[axis]) * invDir;

            if (t0 > t1) std::swap(t0, t1);

            tMin = t0 > tMin ? t0 : tMin;
            tMax = t1 < tMax ? t1 : tMax;
```

```
            if (tMax < tMin) return false;
        }
    }

    outDistance = tMin;
    return true;
}
```

The algorithm handles the special case where the ray is nearly parallel to an axis by checking whether the ray origin lies within the slab. For non-parallel rays, it computes intersection distances with both bounding planes and narrows the valid intersection range by taking the maximum of near intersections and the minimum of far intersections. If the resulting range becomes invalid (near exceeds far), the ray misses the box entirely.

4.4.3 Closest-Hit Selection Logic

The selection system tests the picking ray against all objects in the scene, tracking the closest intersection. This ensures that when multiple objects lie along the ray's path, the frontmost object—the one the user visually clicked on—receives selection.

```
float closestHit = FLT_MAX;
GameObject* hitObject = nullptr;

for (const auto& obj : scene.getObjects())
{
    glm::vec3 aabbMin = obj->getPosition() - obj->getScale() * 0.5f;
    glm::vec3 aabbMax = obj->getPosition() + obj->getScale() * 0.5f;

    float hitDistance;
    if (RayIntersectsAABB(rayOrigin, rayDirection,
                          aabbMin, aabbMax, hitDistance))
    {
        if (hitDistance < closestHit)
        {
            closestHit = hitDistance;
            hitObject = obj.get();
        }
    }
}
```

The loop iterates through all scene objects, testing each against the ray. When an intersection occurs, the distance is compared against the current closest hit. If this object is nearer than any previously tested object, it becomes the new selection candidate. After testing all objects, `hitObject` points to the frontmost intersected object or remains null if the ray missed all geometry.

4.4.4 Input Coordination and Selection State

Object selection is processed only when multiple conditions are met. The engine must be in editor mode, the UI must not be capturing mouse input, gizmo manipulation must be inactive and the left mouse button must be pressed during the current frame. These conditions prevent unintended selections when the user interacts with UI panels or manipulates existing selections using gizmos.

```
if (engineMode == EngineMode::Editor &&
    !uiWantsMouse &&
    !gizmoCapturingMouse &&
    Input::GetMousePressed(GLFW_MOUSE_BUTTON_LEFT))
{
    // Perform raycasting and selection
}
```

The `uiWantsMouse` flag is derived from ImGui's capture state. It indicates whether the mouse is hovering over or interacting with a UI element. The `gizmoCapturingMouse` flag indicates whether the transform gizmo is currently being dragged. Together, these conditions ensure that clicking on the inspector panel or dragging a gizmo handle does not simultaneously select objects in the 3D view.

Selection state is maintained in a vector of selected objects. This supports both single and multiple selection scenarios. When a valid hit occurs, the selection vector is cleared and the newly clicked object is added. When the ray misses all geometry, the selection vector is cleared. This deselects any previously selected objects.

The implementation includes console logging of ray parameters during selection. This aids debugging by displaying the mouse coordinates, ray origin in world space and ray direction vector. This visibility into the coordinate transformation pipeline proved essential during development. It helped diagnose issues such as incorrect matrix inversions or coordinate space mismatches.

4.4.5 Editor-Specific Selection Extensions

Beyond standard game objects, the selection system extends to editor-specific entities, including triggers, force generators and point lights. Each entity type

uses an appropriate intersection test. AABBs are used for box-shaped triggers, while sphere intersection is used for force generators with radial influence and for point light picking volumes.

The system implements selection priority to handle overlapping entities. Force generators have the highest priority, followed by point lights, triggers and finally standard game objects. When multiple entity types intersect the picking ray, the system selects the highest-priority entity. This ensures that editor tools remain accessible even when entities overlap in 3D space.

This raycasting implementation translates 2D mouse input into 3D world-space selections. It correctly handles perspective projection and camera transformations. It also implements efficient AABB intersection testing and coordinates properly with UI and gizmo systems to prevent selection conflicts.

4.5 Editor Mode Architecture

Game engines require dual operational modes: a runtime mode where gameplay systems execute and an editor mode where developers manipulate scene content, inspect object state and debug system behavior. The implementation provides seamless switching between these modes whilst maintaining appropriate separation of concerns and preventing editor functionality from interfering with gameplay.

4.5.1 Mode Switching Mechanism

The engine maintains a global mode state variable that determines which systems are active and how input is processed. Pressing the E key toggles between editor and game modes, triggering a cascade of state changes that reconfigure the engine's behavior.

```
enum class EngineMode { Editor, Game };
EngineMode engineMode = EngineMode::Editor;

if (!ImGui::GetIO().WantCaptureKeyboard &&
    Input::GetKeyPressed(GLFW_KEY_E))
{
    if (engineMode == EngineMode::Editor)
    {
        engineMode = EngineMode::Game;
        // Configure for gameplay...
    }
    else
```

```
    {
        engineMode = EngineMode::Editor;
        // Configure for editing...
    }
}
```

The mode switch checks whether ImGui is capturing keyboard input before processing the E key press, preventing accidental mode changes when typing into text fields or interacting with UI panels. This guard ensures that editor interactions remain isolated from mode control.

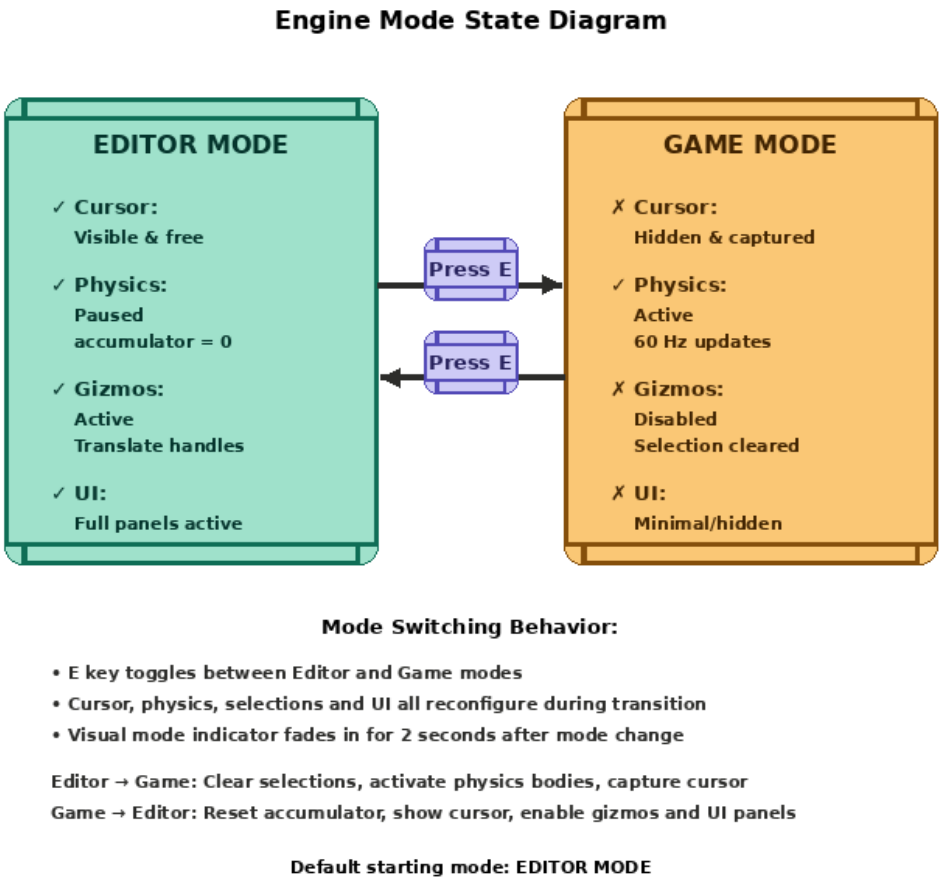

Figure 4.5: Editor and game mode state diagram showing system configuration differences and E key toggle mechanism

4.5.2 Editor Mode Configuration

When entering editor mode, the system performs several configuration changes to enable manipulation and inspection tools. The mouse cursor becomes visible and freely moveable, allowing interaction with ImGui panels and selection of scene objects through clicking. The camera controller receives input events, enabling orbital and free-flight navigation around the scene.

```
engineMode = EngineMode::Editor;
glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_NORMAL);
Input::SetCameraController(&cameraController);
```

Editor mode displays the current mode state prominently on screen for a brief duration, providing immediate visual feedback that the mode transition occurred successfully. This prevents confusion about which systems are active and what input will accomplish.

Selection state is preserved when entering editor mode, allowing developers to resume manipulation of previously selected objects. However, physics simulation updates are prevented from executing as the accumulator is reset to zero each frame rather than being consumed by physics steps.

```
if (engineMode == EngineMode::Game)
{
    while (accumulator >= fixedDt)
    {
        physics.update(fixedDt);
        accumulator -= fixedDt;
    }
}
else
{
    // In editor mode, discard accumulator
    accumulator = 0.0;
}
```

This approach prevents physics from "catching up" when transitioning back to game mode. Without this reset, accumulated time during editor sessions would cause a burst of rapid physics updates upon mode transition, producing unexpected object movements.

4.5.3 Game Mode Configuration

Transitioning to game mode reconfigures the engine for gameplay rather than development. The mouse cursor is hidden and captured, restricting it to the window and preventing it from moving onto other monitors or applications. This cursor capture enables first-person camera control where mouse movement directly rotates the view.

```
engineMode = EngineMode::Game;
glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);
Input::SetCameraController(nullptr);
```

Setting the camera controller to null disconnects editor camera navigation, preventing WASD movement and mouse-look from affecting the view. In game mode, camera control is delegated to gameplay systems rather than editor tools.

All editor selections are cleared when entering game mode, removing highlights and preventing gizmo rendering. This ensures that editor-specific visual elements do not appear during gameplay.

```
selectedObjects.clear();
selectedTrigger = nullptr;
selectedForceGenerator = nullptr;
selectedPointLight = nullptr;
```

Physics bodies are explicitly activated when entering game mode, ensuring that objects which may have been sleeping due to inactivity wake up and respond to forces immediately. This prevents the jarring behavior where objects appear frozen for several frames after mode transition.

```
for (auto& obj : scene.getObjects())
{
    if (obj->hasPhysics())
        obj->getRigidBody()->activate(true);
}
```

4.5.4 Interactive Transform Gizmos

Editor mode provides interactive gizmos—visual handles overlaid on selected objects that enable intuitive manipulation through mouse dragging. The gizmo system renders coloured axis lines representing the X, Y, and Z axes. This allows developers to translate objects along constrained directions using click-and-drag interactions.

The gizmo implementation projects world-space axis endpoints into screen space. This enables hover detection in two-dimensional pixel coordinates. When the mouse cursor approaches an axis line within a defined threshold distance, that axis is highlighted to indicate it can be selected.

```
glm::vec3 originW = selectedObject->getPosition();
float distToCam = glm::length(camera.getPosition() - originW);
float axisLenWorld = glm::clamp(distToCam * 0.15f, 0.5f, 6.0f);

glm::vec3 xEndW = originW + glm::vec3(1,0,0) * axisLenWorld;
glm::vec2 xEndS;
worldToScreen(xEndW, view, proj, fbW, fbH, xEndS);

float dist = distancePointToSegment2D(mousePos, originS, xEndS);
if (dist < threshold) hotAxis = Axis::X;
```

Axis length scales based on camera distance. This ensures that gizmos maintain a consistent visual size on screen, regardless of how far the object is from the camera. It prevents gizmos from becoming too small on distant objects or excessively large on nearby ones.

When an axis is clicked, dragging begins. The system constructs a drag plane that passes through the object's position. This plane is oriented perpendicular to both the selected axis and the camera's forward vector. It provides a stable surface for computing drag positions. As the mouse moves, a ray is cast from the cursor position and intersected with this plane, producing a world-space hit point.

```
glm::vec3 axis = axisDir(activeAxis);
glm::vec3 perpVec = /* most perpendicular camera vector */;
glm::vec3 planeNormal = glm::normalize(glm::cross(axis, perpVec));

glm::vec3 rayOrigin, rayDir;
buildMouseRay(window, fbW, fbH, camera, rayOrigin, rayDir);

glm::vec3 hitPoint;
rayPlaneIntersection(rayOrigin, rayDir, objectPos, planeNormal, hitPoint);
```

The delta between the initial hit point (when drag started) and the current hit point provides a world-space movement vector. This delta is projected onto the active axis to extract only the component parallel to that axis, discarding perpendicular motion.

```
glm::vec3 delta = currentHit - dragStartHit;
float distance = glm::dot(delta, axis);
glm::vec3 newPos = dragStartObjPos + axis * distance;
selectedObject->setPosition(newPos);
```

This projection ensures that object movement remains constrained to the selected axis, regardless of mouse movement direction. Dragging diagonally across the screen still results in movement only along the X, Y, or Z axis as appropriate.

The gizmo system integrates with input coordination. It returns a boolean flag indicating whether it is currently capturing mouse input. When dragging, the object selection system skips raycasting. This prevents accidental reselection while manipulating the current object.

Gizmo functionality extends beyond standard game objects to multiple entity types. Triggers, force generators, point lights, and the directional light all support gizmos. This is achieved through overloaded update and draw functions. It enables a consistent manipulation interface across different editor entity types.

4.5.5 Dear ImGui Integration

Dear ImGui provides the user interface framework for editor tools. It renders debug panels, object inspectors, and scene management windows. The immediate-mode paradigm discussed in the technology review eliminates the need for persistent widget state. Instead, the entire UI is regenerated each frame based on current engine data.

ImGui initialisation occurs during engine startup, where a rendering context is established and connected to the GLFW window and OpenGL. Input events are forwarded to ImGui before being processed by engine systems. This allows UI widgets to capture keyboard and mouse input when appropriate.

```
ImGui_ImplGlfw_InitForOpenGL(window, false);
ImGui_ImplOpenGL3_Init("#version 330");

// In input callbacks:
ImGui_ImplGlfw_KeyCallback(window, key, scancode, action, mods);
ImGui_ImplGlfw_MouseButtonCallback(window, button, action, mods);
```

Each frame begins with ImGui frame setup, processes UI logic that generates widgets based on engine state, and concludes with rendering ImGui's draw data over the 3D scene.

```
ImGui_ImplOpenGL3_NewFrame();
```

```
ImGui_ImplGlfw_NewFrame();
ImGui::NewFrame();

// Draw UI panels...
debugUI.draw(uiContext);

ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());
```

Debug panels display real-time engine statistics including frame rate, physics step count and memory usage. The inspector panel shows properties of the selected object, enabling live editing of transform values, physics parameters and rendering settings. Scene management panels provide save and load functionality, allowing developers to persist and restore scene configurations.

The UI system queries the `WantCaptureMouse` and `WantCaptureKeyboard` flags before allowing engine systems to process input, ensuring that clicking on UI buttons does not simultaneously select objects in the 3D view or trigger gameplay actions.

4.6 Scene Management and Serialisation

Scene persistence enables developers to save content and reload it across development sessions. This preserves object configurations, spatial relationships and gameplay parameters. The implementation provides JSON-based serialisation that balances human readability for version control with efficient parsing that maintains referential integrity as scenes are modified.

Scene Serialization Format (JSON)

```
{
  "objects": [
    {
      "id": 2,                                          ← Unique object ID
      "name": "Player",                                 ← Human-readable name
      "tags": ["player"],                               ← Gameplay tags
      "transform": {
        "position": [0.330, 0.500, 1.931],
        "rotation": [0.0, 165.195, -82.982, 0.500],     Transform data:
        "scale": [1.0, 1.0, 0.500]                      Position, rotation,
      },                                                scale vectors
      "physics": {
        "enabled": true,                                Physics config:
        "mass": 1.0,                                    Mass, material,
        "material": "Default"                           enabled flag
      }
    }
  ],
  "triggers": [...],
  "forceGenerators": [...]
}
```

JSON Structure Features:

- 4-space indentation for human readability
- Separate arrays for objects, triggers and force generators
- Quaternion rotation stored as [x, y, z, w]
- Optional fields (specular, normal maps) omitted when empty

Saved to: ../../assets/scenes/scene_test.json

Figure 4.6: Scene serialisation format showing JSON structure for game objects with transform and physics data

4.6.1 JSON Serialisation Format

JSON was selected as the serialisation format due to its text-based nature. The format enables manual editing in standard text editors and straightforward de-

bugging when scene data becomes corrupted.

The scene file structure organises data hierarchically with separate arrays for different entity types. Game objects occupy one array, triggers another and force generators a third, enabling independent iteration during loading without requiring type discrimination.

```
json sceneJson;
sceneJson["objects"] = json::array();
sceneJson["triggers"] = json::array();
sceneJson["forceGenerators"] = json::array();
```

Each game object serialises to a JSON object containing grouped properties. Basic identification data includes a unique ID, human-readable name and shape type. Transform data encodes position, rotation and scale. Rendering configuration stores texture paths for diffuse, specular and normal maps alongside model file paths for mesh-based objects. Physics parameters record mass, material name and collision volume scale.

```
json o;
o["id"] = obj.getID();
o["name"] = obj.getName();
o["shape"] = (int)obj.getShapeType();

o["transform"]["position"] = {
    obj.getPosition().x,
    obj.getPosition().y,
    obj.getPosition().z
};

o["transform"]["rotation"] = {
    obj.getRotation().x,
    obj.getRotation().y,
    obj.getRotation().z,
    obj.getRotation().w
};

o["physics"]["enabled"] = obj.hasPhysics();
o["physics"]["mass"] = mass;
o["physics"]["material"] = obj.getMaterialName();
```

4.6.2 GameObject Deserialisation

Loading a scene begins by clearing all existing objects from the scene graph, preventing accumulation of data from multiple loads. The implementation iterates through the JSON objects array, reconstructing each game object by parsing its serialised state and invoking appropriate spawning functions.

```
clear();  // Remove existing objects

for (const auto& o : sceneJson["objects"])
{
    ShapeType shape = (ShapeType)o["shape"].get<int>();

    glm::vec3 position(
        o["transform"]["position"][0],
        o["transform"]["position"][1],
        o["transform"]["position"][2]
    );

    glm::quat rotation(
        o["transform"]["rotation"][3],  // w
        o["transform"]["rotation"][0],  // x
        o["transform"]["rotation"][1],  // y
        o["transform"]["rotation"][2]   // z
    );
```

Deserialisation requires careful ordering. GLM's quaternion constructor expects the w component first, whereas the serialised data stores values in the order x, y, z, w. The indices are therefore reordered during reconstruction to match GLM's expected parameter sequence.

Object creation branches based on whether physics is enabled and whether the object uses a custom mesh model. Physics-enabled objects invoke a physics-aware spawn function. This creates both rendering and physics representations. Non-physics objects invoke a render-only spawn function, which omits collision shapes and rigid body creation.

```
GameObject* obj = nullptr;

if (physicsEnabled)
{
    float mass = o["physics"]["mass"];
    std::string material = o["physics"]["material"];
```

```
    obj = spawnObject(shape, position, collisionSize,
                      mass, material, texture, specular);
}
else
{
    obj = spawnRenderObject(shape, position, scale,
                            texture, specular);
}
```

Model-based objects require special handling, as they combine mesh geometry with optional physics collision volumes. The loading process distinguishes between visual mesh scale, used for rendering, and physics collision scale, used for bounding volume construction. This allows independent tuning of visual and physical representations.

After object creation, additional properties are applied. Rotation is set explicitly, as spawn functions initialise objects with a default orientation. Object names and tags are restored from serialised data. This enables gameplay systems to locate specific objects through queries.

4.6.3 Editor-Specific Entity Serialisation

Triggers and force generators represent editor-specific entities that exist only during development. They provide debugging tools and support gameplay prototyping. These entities are serialised independently from standard game objects, with type-specific parameter storage.

Triggers encode position, size, and trigger type, along with behaviour-specific data. Teleport triggers store destination coordinates. Speed zone triggers preserve force direction and magnitude. Required tags enable filtering of which objects can activate the trigger.

```
json t;
t["name"] = trigger->getName();
t["type"] = static_cast<int>(trigger->getType());
t["position"] = { pos.x, pos.y, pos.z };
t["size"] = { size.x, size.y, size.z };

if (trigger->getType() == TriggerType::TELEPORT)
{
    glm::vec3 dest = trigger->getTeleportDestination();
    t["teleportDestination"] = { dest.x, dest.y, dest.z };
}
```

Force generators serialise position, radius and strength parameters common to all generator types, then append type-specific data. Wind generators store direction vectors, gravity wells preserve minimum distance thresholds and vortex generators encode rotation axes and pull strength.

The serialisation system handles polymorphic entity types through type discrimination during deserialisation. The stored type enumeration determines which concrete class to instantiate and which parameters to extract from the JSON data.

4.6.4 Object Identity and Reference Management

Each game object receives a unique integer identifier at creation time, assigned sequentially from a global counter. This ID persists through serialisation, enabling external systems to reference specific objects through stable identifiers rather than memory pointers.

```
o["id"] = obj.getID();

// During load:
if (o.contains("name"))
    obj->setName(o["name"]);
```

The current implementation does not preserve or restore object IDs during loading, instead allowing the spawn functions to assign fresh IDs as objects are created. This simplifies the loading process but breaks references if external systems stored object IDs between sessions. Future enhancement would restore original IDs explicitly to maintain referential integrity.

4.6.5 File System Integration

Scene files reside in a dedicated directory structure at `../../assets/scenes/`, organised separately from code and other asset types. The save operation creates parent directories if they do not exist, preventing failures when attempting to write to non-existent paths.

```
std::filesystem::create_directories(
    std::filesystem::path(path).parent_path());

std::ofstream file(path);
if (!file.is_open())
{
    std::cout << "Failed to save scene: " << path << std::endl;
```

```
    return false;
}

file << sceneJson.dump(4);
```

The JSON library's dump function formats output with 4-space indentation. This produces human-readable files that are suitable for manual editing and version control. Compression is not applied, prioritising readability and editability over file size optimisation.

Loading includes error checking for file operations. If a file cannot be opened, the system returns false and logs an error message. This allows calling code to detect failures and take appropriate recovery actions, rather than silently continuing with invalid data.

4.6.6 Scene Management UI Integration

The scene manager panel provides UI controls for creating, saving, loading, renaming and deleting scene files. The interface scans the scene directory once when first displayed, populating a dropdown list with available scene files.

```
static std::vector<std::string> sceneFiles;

if (std::filesystem::exists(sceneFolder))
{
    for (auto& entry : std::filesystem::directory_iterator(sceneFolder))
    {
        if (entry.path().extension() == ".json")
        {
            sceneFiles.push_back(entry.path().stem().string());
        }
    }
}
```

Scene operations are restricted to editor mode through conditional UI disabling. Save, load, rename and delete buttons become non-interactive in game mode. This prevents accidental modification of scene data during gameplay testing. The implementation uses ImGui's `BeginDisabled` and `EndDisabled` functions to grey out controls and ignore input on disabled elements.

```
if (engineMode != EngineMode::Editor)
    ImGui::BeginDisabled();
```

```
if (ImGui::Button("Save Scene"))
{
    scene.saveToFile(fullPath);
}

if (engineMode != EngineMode::Editor)
    ImGui::EndDisabled();
```

Delete operations use a confirmation popup to prevent accidental file removal. Clicking the delete button opens a modal dialog that requires explicit yes/no confirmation before deleting the file from disk. This prevents accidental clicks when the user intends to load or rename a scene.

The new scene operation clears the current scene and spawns a default ground plane. This provides a minimal starting point for scene authoring. It ensures that developers begin with a stable foundation, rather than an empty space where objects would fall indefinitely.

Chapter 5

System Evaluation

This chapter evaluates the framework implementation against the five objectives established in the introduction. It presents evidence of successful achievement, alongside a discussion of limitations and challenges encountered during development. Each objective is assessed based on functional correctness, performance characteristics, and integration with other engine subsystems.

5.1 Objective 1: Fixed-Timestep Game Loop

The fixed-timestep game loop implementation successfully achieves deterministic physics simulation independent of rendering frame rate. Runtime measurements confirm that the system maintains the target 60 Hz physics update rate whilst accommodating variable rendering performance.

5.1.1 Timing Accuracy

The debug statistics panel displays real-time performance metrics during engine operation. Frame rate measurements consistently show values between 59.8 and 60.1 frames per second, with delta time values holding steady at 0.0167 seconds (16.67 milliseconds). This precision confirms that the chrono-based timing system provides sufficient resolution for accurate frame time measurement.

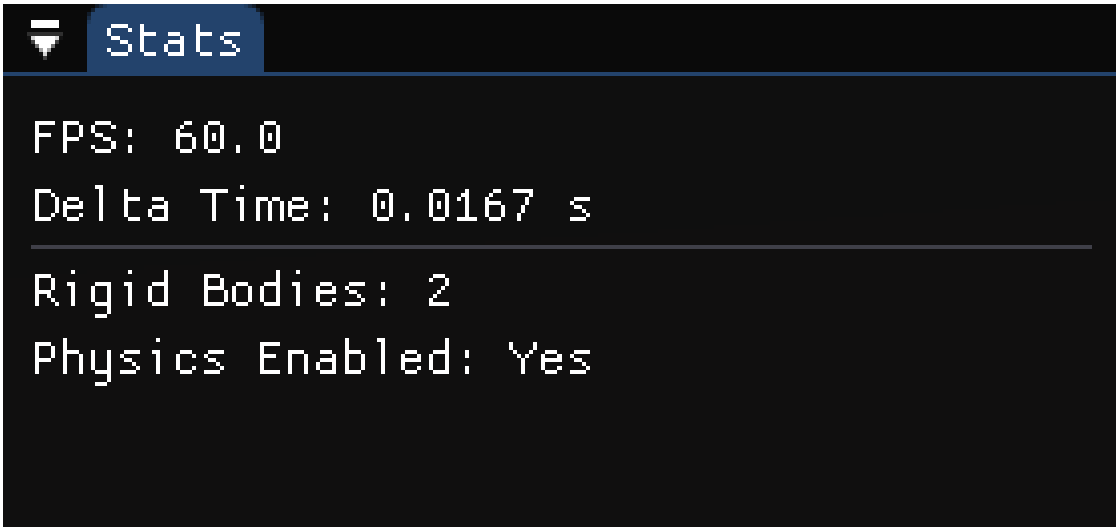

Figure 5.1: Statistics panel showing fixed-timestep performance (FPS: 60.0, Delta Time: 0.0167s) and physics state (2 rigid bodies, physics enabled)

Console logging verifies that physics updates execute at exactly 60 steps per second during gameplay, regardless of rendering frame rate variations. When rendering performance drops due to complex scenes or system load, the accumulator pattern ensures that physics continues advancing in fixed increments. This prevents the frame-rate-dependent behaviour associated with variable timestep implementations.

5.1.2 Spiral of Death Prevention

The 250-millisecond frame time cap successfully prevents spiral-of-death scenarios during temporary performance hitches. Testing with artificially induced delays confirmed that the system degrades gracefully to slow motion, rather than hanging indefinitely. When frame processing exceeds the fixed timestep duration, the clamped frame time prevents unbounded accumulator growth. This allows the system to skip forward and recover once performance improves.

5.1.3 Editor Mode Physics Pausing

The accumulator reset mechanism correctly prevents physics simulation during editor mode. Console output displays “Physics steps per second: 0” when operating in editor mode. This confirms that physics updates are skipped while developers

manipulate scene content. This isolation ensures that authored object positions remain stable during editing, rather than being overwritten by physics simulation.

Transitioning from editor to game mode correctly activates all physics bodies. This prevents the frozen-object behaviour that would occur if sleeping bodies were not explicitly awakened. The implementation successfully balances deterministic simulation timing with editor-friendly behaviour.

5.1.4 Performance and Stress Testing

Additional testing was conducted to evaluate the performance characteristics of the physics system under varying workloads. Initial validation confirmed that the engine maintained a stable 60 FPS when operating with a low number of rigid bodies. This demonstrated that the fixed-timestep implementation performs reliably under normal conditions.

A stress test was then performed by spawning up to 1000 dynamic rigid bodies. Under this scenario, performance degraded significantly and frame rates dropped to approximately 4 FPS. This indicated that the physics simulation introduced substantial computational overhead when handling a large number of active bodies.

To isolate the source of the performance bottleneck, an equivalent test was conducted using static rigid bodies (mass = 0). In this configuration, performance improved considerably with frame rates maintaining between 40 and 60 FPS. This confirms that rendering was not the primary limiting factor and that the performance degradation was primarily caused by physics simulation.

Further analysis was carried out by monitoring the number of active rigid bodies during simulation. Results showed that following initial collision resolution, the majority of bodies transitioned into a sleeping state. The number of active bodies decreased from approximately 750 to as few as 2–5. This demonstrates that the Bullet physics engine's sleep optimisation is functioning correctly and reduced unnecessary computations for inactive objects.

The results indicate that the physics system is stable and correctly integrated. Performance degradation occurs primarily during high-collision scenarios involving many simultaneously active dynamic bodies. Performance recovers significantly once objects settle and enter a sleeping state.

5.2 Objective 2: Editor Mode with Interactive Debugging Tools

The editor mode implementation provides comprehensive development tools while maintaining a clear separation from gameplay systems. The E key toggle enables

seamless transitions between modes. It reconfigures input handling, physics behaviour and UI visibility as appropriate.

5.2.1 Mode Switching Functionality

Mode transitions execute correctly, applying the necessary configuration changes to cursor visibility, input routing and system activation. When entering game mode, the cursor becomes hidden and captured, enabling first-person camera control. Editor selections are cleared automatically, preventing gizmo rendering during gameplay. Physics bodies are also activated to ensure immediate response to forces and collisions.

Returning to editor mode restores cursor visibility and reconnects the camera controller to input events. The physics accumulator is reset to prevent catch-up behaviour. A visual mode indicator provides immediate feedback, displaying "EDITOR MODE" or "GAME MODE" text that fades over two seconds.

5.2.2 Dear ImGui Integration

The Dear ImGui integration successfully provides real-time debugging interfaces without requiring modifications to the rendering pipeline. Debug panels display frame timing statistics, physics metrics, and system state. The inspector panel presents selected object properties, including transform data, physics configuration, and material assignments. This enables live editing of object parameters.

The scene manager panel provides file operations for saving, loading, renaming, and deleting scene files. The trigger editor and force generator panels support runtime creation and configuration of gameplay systems. All UI panels respect editor and game mode boundaries. Modification controls are disabled during gameplay to prevent accidental state corruption.

5.2.3 Interactive Transform Gizmos

The gizmo system enables intuitive object manipulation through mouse-based dragging. Coloured axis lines—red for X, green for Y, and blue for Z—are rendered over selected objects, providing a clear visual indication of available movement directions. Hover detection highlights axes when the mouse cursor approaches within a defined threshold distance. Clicking an axis initiates constrained dragging along the selected direction.

The world-to-screen projection maintains a consistent gizmo size regardless of camera distance. This prevents gizmos from becoming too small on distant objects or excessively large on nearby ones. Ray-plane intersection calculations produce

stable drag behaviour, ensuring that object movement remains constrained to the selected axis, even during diagonal mouse movement.

Gizmo functionality extends to multiple entity types, including standard game objects, triggers, force generators and lights. This provides a consistent manipulation interface across different editor tools.

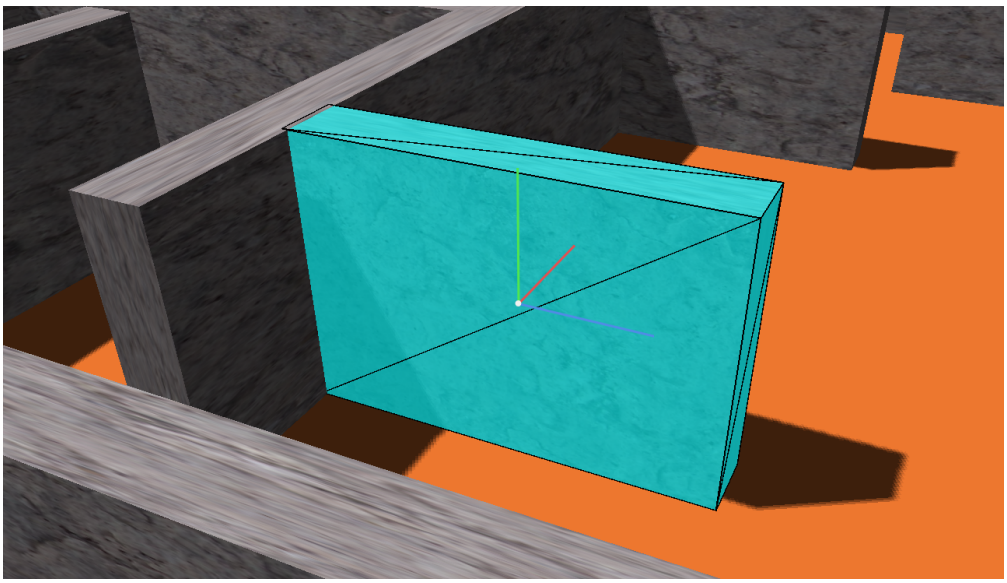

Figure 5.2: Interactive transform gizmo showing red (X), green (Y) and blue (Z) axes for axis-constrained object manipulation

5.3 Objective 3: Modular Input System with State Tracking

The input system successfully tracks three temporal categories of input state, integrates cleanly with GLFW's callback model and coordinates correctly with Dear ImGui to prevent input conflicts.

5.3.1 State Tracking Accuracy

The three-state tracking mechanism (pressed, held, released) operates correctly, enabling distinct gameplay mechanics based on input timing. Continuous movement systems query the held state for WASD navigation. Single-fire actions query the pressed state to trigger once per key press. Release-triggered mechanics use the released state for features such as charging attacks or aim-down-sights systems.

The per-frame state reset functions correctly, clearing pressed and released flags at the beginning of each frame before new input events are processed. This prevents stale state from affecting subsequent frames and ensures that single-frame events trigger exactly once.

5.3.2 ImGui Input Coordination

Input forwarding to Dear ImGui prevents UI click-through behaviour, which would otherwise allow simultaneous interaction with UI panels and 3D scene objects. Checking `WantCaptureMouse` and `WantCaptureKeyboard` before processing gameplay input ensures that clicking UI buttons does not also select objects. It also prevents typing in text fields from triggering hotkey actions.

The implementation also resolves keyboard input conflicts. For example, typing 'E' into name fields would otherwise trigger editor/game mode switching. Checking `WantCaptureKeyboard` before processing the E key press prevents this issue. This allows text input to function correctly while preserving hotkey behaviour when the UI is not active.

5.3.3 Camera Controller Integration

Mouse delta tracking provides smooth camera rotation through the connected camera controller. The first-mouse guard prevents the jarring camera snap that would occur from calculating delta against uninitialised position values. Right-click-and-drag functionality enables editor-style camera manipulation whilst leaving left-click available for object selection.

5.4 Objective 4: Component-Based Object Interaction and Selection System

The raycasting and object selection system successfully translates screen-space mouse clicks into world-space object selections, handles perspective projection correctly and coordinates with UI and gizmo systems to prevent selection conflicts.

5.4.1 Coordinate Transformation Accuracy

Console logging of ray parameters during selection events confirms correct coordinate space transformations. For example, a click at screen coordinates (570, 325) produces a ray origin that matches the camera position at (8.68578, -31.9531)

in world space. It also produces a normalised direction vector of (0.0782385, -0.171222, 0.982121). The direction vector has a magnitude of approximately 1.0, confirming correct normalisation within floating-point precision.

The transformation pipeline correctly handles framebuffer-to-NDC conversion, including the inversion of the y-axis between screen space (origin at the top-left) and NDC space (origin at the bottom-left). Applying the inverse projection and view matrices produces rays that pass through the selected screen position. This is verified visually, as objects under the cursor are correctly highlighted when clicked.

```
[Editor Ray]
  Mouse: (484, 672)
  Origin: (13.7896, 3.8454, -0.409409)
  Direction: (-0.814394, -0.467114, 0.344336)
```

Figure 5.3: Console output showing editor raycasting calculations with screen coordinates (484, 672), world-space origin and normalised direction vector.

5.4.2 AABB Intersection Testing

The slab method implementation correctly identifies ray–box intersections. It handles edge cases such as rays parallel to axis-aligned planes and intersections occurring behind the ray origin. Closest-hit tracking ensures that, when multiple objects lie along the ray path, the frontmost object is selected. This matches user expectations of interacting with visible geometry.

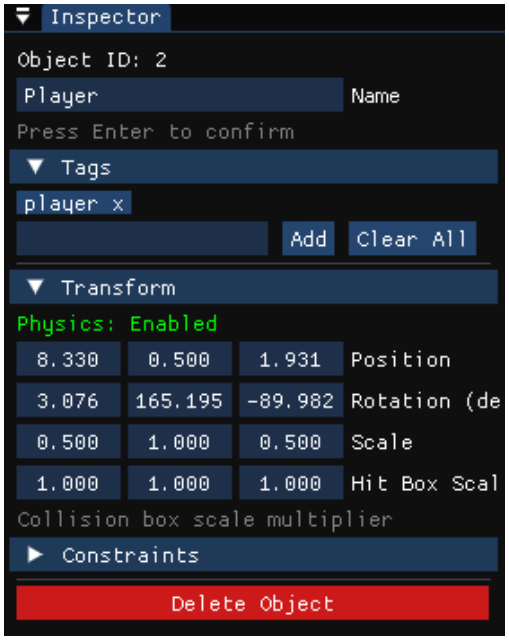

Figure 5.4: Inspector panel displaying properties of selected object (ID: 2, Name: Player) including transform data, physics configuration and tag system

5.4.3 Inspector Panel Integration

Selected objects display their properties in the inspector panel, including object ID, name, tags, transform data, physics configuration, and material assignments. The panel supports live editing of transform values, with changes immediately reflected in the 3D view. The presence of Object ID 2 in the inspector screenshot confirms that the identification system correctly assigns unique integer identifiers to all objects.

The delete object functionality correctly removes selected entities from the scene while maintaining the integrity of the selection vector. Multiple selection is also supported, allowing simultaneous manipulation of object groups. Pressing the Delete key removes all selected objects in a single operation.

5.5 Objective 5: Scene and Game Object Management System

The scene management implementation provides robust serialisation and deserialisation of complex object hierarchies whilst maintaining referential integrity and supporting editor workflow integration.

5.5.1 JSON Serialisation Correctness

Scene files are serialised to JSON with 4-space indentation. This enables manual editing and produces meaningful version control diffs. The hierarchical structure separates game objects, triggers and force generators into distinct arrays. This simplifies type-specific iteration during loading.

Object serialisation captures all necessary state. This includes unique IDs, names, tags, shape types and transform data such as position, rotation (stored as a quaternion) and scale. It also includes rendering configuration, such as texture paths for diffuse, specular and normal maps, as well as model file paths. Physics parameters are also stored, including an enabled flag, mass, material name and collision scale. The format successfully represents both primitive shapes and complex mesh-based objects.

5.5.2 Load/Save Cycle Integrity

Round-trip testing confirms that scenes can be saved and reloaded without data loss. Object positions, rotations and scales match their authored values after load. Physics-enabled objects correctly reconstruct rigid bodies with appropriate mass and material properties. Texture assignments persist correctly, with diffuse, specular and normal maps loading from stored paths.

Triggers and force generators deserialise with type-specific parameters intact. Teleport triggers restore destination coordinates, speed zones preserve force vectors and magnitudes, and force generators correctly reconstruct radius, strength and type-specific data such as wind direction or vortex axes.

5.5.3 Editor Workflow Integration

The scene manager UI provides file operations without requiring command-line interaction or manual file manipulation. A dropdown list displays available scene files from the designated directory, enabling quick switching between authored content. Save, load, rename and delete operations execute correctly. Appropriate safeguards are in place, including confirmation popups for deletion and restrictions that prevent saving during gameplay.

Keyboard shortcuts, such as F5 for save and F9 for load, enable rapid iteration during development. The new scene operation correctly clears existing content and spawns a default ground plane.

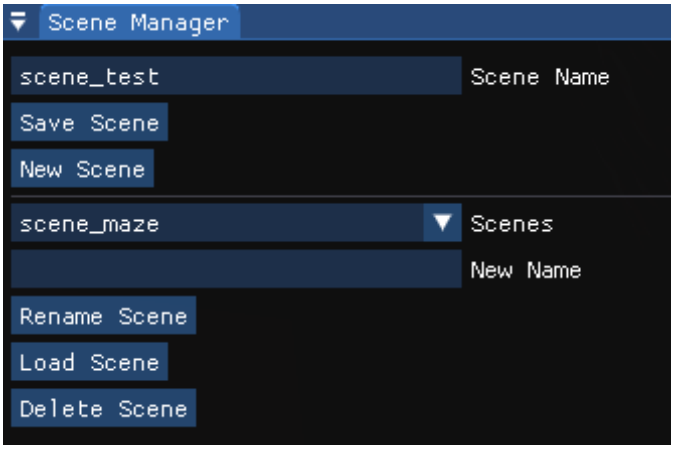

Figure 5.5: Scene Manager UI panel providing save, load, rename and delete operations with dropdown list of available scenes

5.6 Challenges and Limitations

Development of the framework component encountered several architectural and integration challenges that required iterative refinement and, in some cases, significant refactoring.

5.6.1 ImGui Ownership Refactoring

The initial implementation embedded ImGui lifecycle management within the Renderer module, creating tight coupling between the UI framework and the graphics system. This violated separation of concerns as the renderer should only handle visual output rather than manage unrelated subsystems. The architecture was refactored to move ImGui initialisation, frame management and cleanup to the Engine module. This better reflects the role of lifecycle management alongside other core systems.

This refactoring highlighted the importance of careful architectural planning. Although it ultimately improved system organisation. It consumed development time that could have been avoided with more thorough upfront design.

5.6.2 UI Click-Through Bug

Early versions of the editor suffered from input conflicts where mouse clicks on UI panels would simultaneously select objects in the 3D scene behind them. This occurred because the raycasting selection system processed all left-clicks without

checking whether the UI had already handled the event. The fix required implementing `WantCaptureMouse` checks before invoking selection logic. This ensured UI interactions remain isolated from scene manipulation.

This issue demonstrated the subtle complexities of integrating immediate-mode GUI frameworks with interactive 3D environments. The solution was straightforward once understood but discovering the root cause required careful debugging of event propagation through multiple system layers.

5.6.3 Keyboard Input Conflicts

A similar input conflict affected keyboard handling. Typing characters into text fields would trigger unintended hotkey actions. For example, entering 'E' into an object name field would inadvertently switch between editor and game modes, interrupting the naming workflow. The solution involved checking `WantCaptureKeyboard` before processing mode toggles and other hotkey bindings. This prevents text input from triggering command shortcuts.

This issue highlighted the need for comprehensive input validation beyond simple key state queries. Robust input handling must account for context. The same key press should produce different outcomes depending on whether UI text fields are active or whether the user is interacting with the 3D environment.

5.6.4 Editor/Game Mode State Management

Managing state transitions between editor and game modes proved more complex than initially anticipated. Early implementations exhibited physics "catch-up" behaviour. Time accumulated during editor sessions caused a burst of rapid physics updates when switching to game mode. As a result, objects would jerk violently or pass through collision geometry as the system attempted to process multiple timesteps in a single frame.

The solution required explicitly resetting the accumulator to zero while in editor mode, preventing time accumulation. Physics bodies also needed to be explicitly activated when entering game mode to ensure that sleeping bodies responded immediately to forces. Selection state was cleared during mode transitions to prevent gizmo rendering during gameplay.

These state management challenges highlight the architectural complexity of dual-mode engines, where development tools and gameplay systems coexist within a single application. Each mode transition represents a substantial reconfiguration of multiple subsystems. This requires careful orchestration to maintain correct behaviour.

Chapter 6

Conclusion

This dissertation has presented the design, implementation and evaluation of a modular framework architecture for a 3D game engine built using C++, OpenGL, GLFW and Bullet Physics. The framework serves as the foundational layer, orchestrating timing, input management, scene organisation and editor functionality. It provides the infrastructure upon which the rendering and physics subsystems operate.

Through systematic development guided by five specific technical objectives, the project has produced a functional engine framework capable of supporting real-time interactive 3D applications.

6.0.1 Achievement of Objectives

The five objectives established in the introduction have been successfully realised through careful architectural design and iterative implementation:

Objective 1: Fixed-Timestep Game Loop was achieved through the implementation of an accumulator-based timing system that maintains deterministic 60 Hz physics updates, independent of rendering frame rate. Performance measurements confirmed consistent delta time values of 16.67 milliseconds and physics step rates of exactly 60 updates per second during gameplay. The spiral-of-death prevention mechanism handles temporary performance degradation. Additionally, the editor mode accumulator reset correctly prevents physics simulation during scene authoring.

Objective 2: Editor Mode with Interactive Debugging Tools was achieved through a comprehensive dual-mode system that enables seamless transitions between development and gameplay contexts. The Dear ImGui integration provides real-time inspection of engine state without tightly coupling UI logic to the rendering system. Interactive transform gizmos with axis-constrained dragging enable intuitive object manipulation. Debug panels expose performance metrics

and system configuration, while a visual mode indicator clearly communicates the current engine state to developers.

Objective 3: Modular Input System with State Tracking was achieved through a callback-based architecture that accurately tracks pressed, held, and released input states across frames. The system integrates effectively with GLFW's event model while providing a simplified query API for gameplay code. Per-frame state reset prevents stale data from affecting frame logic, and Dear ImGui input coordination eliminates conflicts between UI interaction and scene manipulation.

Objective 4: Component-Based Object Interaction and Selection System was achieved through raycasting calculations that transform screen-space mouse coordinates into world-space selection rays. The slab method for AABB intersection testing efficiently identifies clicked objects, while closest-hit logic ensures that foreground objects are prioritised. The inspector panel provides comprehensive property editing, and selection state integrates correctly with gizmo manipulation and deletion operations.

Objective 5: Scene and Game Object Management System was achieved through a JSON-based serialisation system that persists complex object hierarchies with full fidelity. The format captures transform data, rendering configuration, and physics parameters, while remaining human-readable for version control workflows. The scene manager UI provides file operations for creating, saving, loading, and deleting scenes, with keyboard shortcuts enabling rapid iteration during development.

6.0.2 Insights and Lessons Learned

The development process yielded valuable insights into real-time systems architecture and the challenges of building modular engine frameworks:

Architectural Planning Matters: The ImGui ownership refactoring demonstrated the cost of insufficient initial design. Moving ImGui lifecycle management from the Renderer to the Engine module required substantial code changes. This could have been avoided with more thorough upfront architectural planning. This experience reinforces that time invested in design helps prevent costly refactoring later.

Integration Complexity in Dual-Mode Systems: Managing state transitions between editor and game modes proved more complex than initially anticipated. Each transition affects cursor visibility, input routing, physics activation and selection state simultaneously. Achieving robust mode switching required careful orchestration across multiple subsystems. This highlights that dual-mode engines are inherently more complex than single-mode alternatives.

Input Context Awareness is Essential: The UI click-through and keyboard conflict issues demonstrated that robust input handling requires context awareness

beyond simple state queries. The same mouse click or key press must produce different outcomes depending on whether UI elements are active. This necessitates coordination mechanisms such as `WantCaptureMouse` and `WantCaptureKeyboard`, which explicitly communicate input capture state between systems.

Empirical Testing Validates Theory: While coordinate space transformations and ray intersection mathematics can be understood theoretically, correct implementation required empirical validation through console logging and visual debugging. The accuracy of the raycasting system was ultimately confirmed through observed behaviour during interactive testing, rather than purely through mathematical proof.

6.0.3 Limitations and Future Work

Several limitations and opportunities for enhancement were identified during evaluation:

The raycasting selection system tests every object each frame. This may become a performance bottleneck in scenes containing large numbers of objects. Implementing spatial partitioning structures, would improve selection performance in more complex scenes.

The gizmo system currently supports only translation. Rotation and scale gizmos were planned but not implemented due to time constraints. Adding these features would enable full transform editing without relying on numerical input in the inspector panel.

The scene management system lacks undo/redo functionality and prefab support, both of which would improve workflow efficiency. Implementing these features would require command pattern infrastructure for reversible operations, as well as reference management systems for instanced content.

6.0.4 Broader Impact and Transferable Skills

Beyond the specific technical achievements documented in this dissertation, the project has provided valuable experience in real-time systems development, software architecture and collaborative engineering practices. Challenges such as maintaining deterministic physics timing, managing object lifecycles and coordinating subsystem integration are representative of those encountered in real-time simulation systems, not just game engines.

Working within a modular architecture with clearly defined interfaces between components reflects professional software engineering practices. The need to document interfaces, coordinate integration points and maintain architectural boundaries provided practical experience in large-scale software organisation. These skills

extend beyond game development to any domain requiring component-based system design.

6.0.5 Final Remarks

This project has demonstrated that building a game engine from scratch provides deep insight into real-time systems architecture, beyond what can be gained from using existing engines. The framework achieves its stated objectives while revealing the underlying complexities of timing systems, input coordination and dual-mode architectures that commercial engines often abstract away.

The modular approach to engine architecture proved effective for parallel development. It enabled three team members to make independent progress while maintaining system coherence through well-defined interfaces. The framework, acting as the integration layer, successfully coordinates rendering and physics subsystems. It also provides comprehensive debugging and authoring tools for scene development.

Looking forward, the framework provides a strong foundation for further engine development. Its modular design supports the extension of existing systems and the addition of new features without requiring major redesign. The insights gained—particularly in timing determinism, input coordination and state management—are transferable to future work in real-time systems across various domains.

The progression from basic window initialisation to a fully functioning game engine with editor tools has been both challenging and educational. It highlights the value of low-level engine development as a learning experience. The framework demonstrates that the systematic application of software engineering principles, supported by research and validated through empirical testing, can produce effective real-time system architectures, even within the complex domain of interactive 3D game engines.

Appendix A

Source Code

The full implementation is available at:

`https://github.com/ATU-Game-Engine/GameEngine`

Bibliography

[1] R. Nystrom. (2014) Game programming patterns. [Online]. Available: https://gameprogrammingpatterns.com/

[2] O. Cornut. (2024) Dear imgui: Bloat-free graphical user interface for c++ with minimal dependencies. [Online]. Available: https://github.com/ocornut/imgui

[3] Scratchapixel. (2024) Ray tracing: Generating camera rays. [Online]. Available: https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-generating-camera-rays/generating-camera-rays.html

[4] (2024) Gameobject. Unity Technologies. [Online]. Available: https://docs.unity3d.com/ScriptReference/GameObject.html

[5] G. Fiedler. (2004) Fix your timestep! [Online]. Available: https://gafferongames.com/post/fix_your_timestep/

[6] *GLFW Documentation*, GLFW, 2024. [Online]. Available: https://www.glfw.org/documentation.html

[7] (2024) Physics.raycast. Unity Technologies. [Online]. Available: https://docs.unity3d.com/ScriptReference/Physics.Raycast.html

[8] (2017) The json data interchange syntax. ECMA International. [Online]. Available: https://www.json.org/json-en.html